3. Design and implementation

The Cartesian plotter constrains the workspace, pen state and motion required from the software. The design therefore starts with hardware, then follows page segmentation, text/writer classification and handwriting reproduction, before describing their integration.

Implementation-specific equations, parameter values and pseudocode describe the project's CleanPipeline3 source [1]; borrowed methods are cited at their point of use. Worked numerical examples are derived here unless identified as recorded measurements.

3.1 Design summary

This section summarises the project design tasks and how they were implemented (see Table 1). The tasks follow the deliverables and student design work in the approved Project Proposal. Completion refers to the implementation of a task; compliance with a numerical system specification is established separately by the qualification tests in the Results section. [1, project design summary]

Deliverable or task	**Implementation**	**Completion and report location**
Investigation of methods	Compared recognition, conditioning and reproduction approaches.	Completed; Literature Review; Sections 3.5, 3.6.
Page capture and image conditioning	Camera arrangement, illumination normalisation, ink extraction and orientation correction.	Completed; Sections 3.3.6 and 3.4.
Separation of handwritten text	Line detection, curved boundaries and ink ownership; later transcript-constrained glyph localisation.	Completed; Sections 3.4, 3.6.2.
Text and writer classifiers	Convolutional features, sequence recognition and writer identification.	Completed; Section 3.5.
Network training implementations	NumPy calculations and framework training; selected weights are framework-trained.	Completed; Sections 3.5.5–3.5.7.
Writer-specific trajectory maps and storage	Aligned glyph extraction, skeleton tracing, variant filtering and persistent style profiles.	Completed; Sections 3.6.2–3.6.5.
Handwriting synthesis and G-code	Text layout, glyph assembly, candidate selection and conversion to machine coordinates.	Completed; Sections 3.6.6–3.6.8.
Gantry, pen mechanism and motion control	Custom mechanical assembly, two-axis drive, pen switching, end stops and calibration.	Completed; Section 3.3.
Processing and motion integration	Server connects capture, processing, stored profiles and execution.	Completed; Section 3.7.
Subsystem verification	Numerical checks and offline image/path experiments; formal integrated qualification is reported in Results.	Completed; Sections 3.5.7, 3.4.6; Appendix B.

Table 1. Design summary. [2, 1]

3.2 System block diagram

The complete system connects page capture, handwriting interpretation and physical reproduction. The camera supplies an image to the ODROID N2+, while the gantry receives motion commands from the same computer. Between these interfaces, line images, recognised strings, writer probabilities and pen trajectories are kept as different representations. This makes it possible to inspect the output of one stage before it is used by the next. [1, web/server.py]

Figure 1 shows the hardware connections. The two stepper drivers supply the winding currents; the ODROID supplies STEP and DIR signals rather than motor power through its GPIO pins. Four switches indicate the axis limits and a fifth indicates the pen state. The pen motor is controlled through its own switching circuit. Camera acquisition, inference and job execution are coordinated by the server. [1, project implementation]

Figure requested: Labelled system diagram distinguishing motor power, logic signals, switch feedback and camera data.

`design-hardware-overview.pdf`

Figure 1. Hardware architecture of the handwriting robot. [2, 1]

Figure 2 separates the three operating modes. During preparation, labelled line images train the text and writer models, and the text model is also used to extract a writer's glyphs from lines with known transcripts. The resulting weights and style profiles are stored for later use. During classification, the input page is conditioned and separated into lines; the reader produces text and the writer classifier produces a probability distribution over the trained writers. During reproduction, digital text and a selected profile are converted into a trajectory, then G-code, and finally GPIO pulses and pen commands. [1, project implementation]

Recognition and reproduction share models, but they do not perform the same operation. Recognition estimates unknown text from an image. Glyph extraction uses text that is already known to estimate where its characters occur. Reproduction arranges the extracted pen paths into new text and uses the reader and writer classifier to select among candidate renderings. Keeping these roles separate is important: a candidate-selection score indicates how a model judges the generated image, whereas an independently assessed physical page establishes what the complete machine actually reproduced. [1, web/server.py]

The final design reads complete lines rather than requiring isolated letters at FU 2.2. Character boundaries are instead recovered with known transcripts during profile construction. This avoids committing to uncertain cuts before recognising connected handwriting [1, recognise.py, PaperCRNNNumpy; style.py, ExtractLineGlyphs].

The proposal's functional units remain identifiable in this architecture. Camera acquisition and request handling implement FU 1; conditioning and line preparation implement FU 2.1–2.2; the two trained networks implement classification FU 2.3. Offline weight training corresponds to FU 2.4, while glyph extraction, profile storage and glyph retrieval implement FU 2.5–2.7. Trajectory conversion and the motion interpreter implement FU 2.8, followed by the axis drives, pen holder and lift in FU 3.1–3.4. Training and profile preparation produce stored artefacts before request-time recognition or writing; they are not repeated for each page. [1, web/server.py]

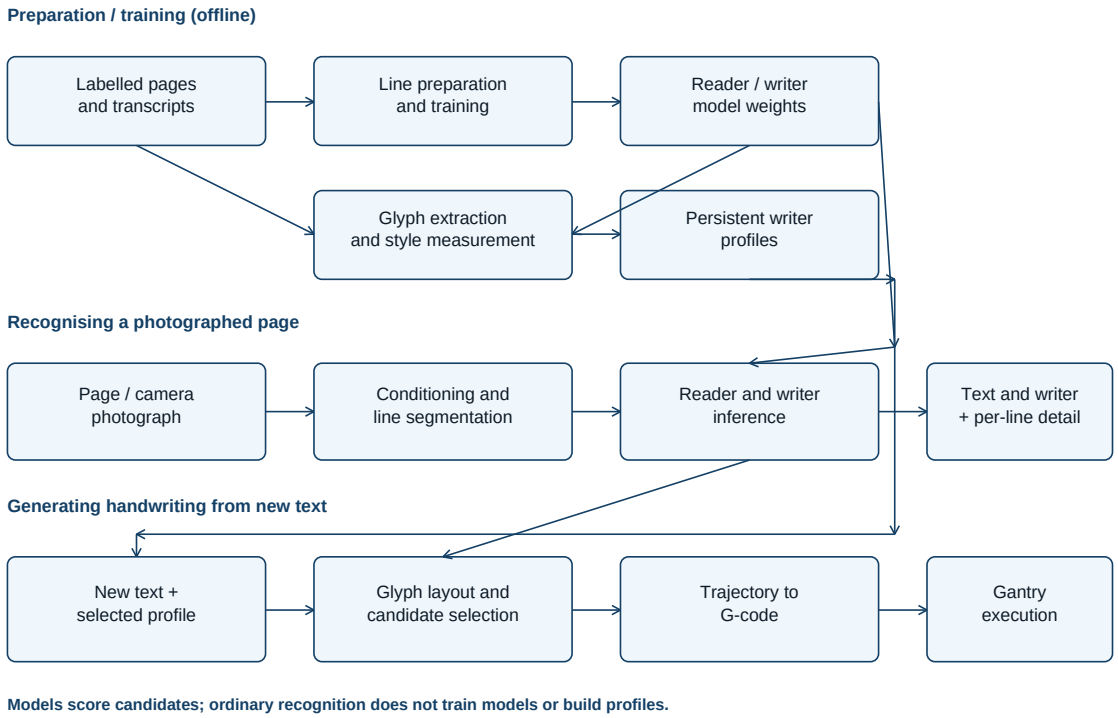


Figure 2. Processing architecture and the stored information shared by the operating modes. [2, 1]

The interface between software and mechanics is G-code. It describes motion in millimetres and pen state without requiring the synthesiser to know GPIO pin numbers. Conversely, the motion interpreter can draw a geometric test file without loading a handwriting model. This separation supports commissioning the gantry independently and localising errors to image processing, trajectory generation or physical execution. [1, web/server.py]

3.3 Hardware design

The hardware must hold the paper and pen securely, reach the required writing area, and convert a stored trajectory into repeatable motion. The design therefore combines a Cartesian belt drive with a separately actuated pen mechanism. The electrical interface is kept simple enough for direct control from the ODROID, while calibration relates software coordinates to the travel of the assembled machine. [2, hardware clarifications][1, reproduce.py]

3.3.1 Plotter mechanics and mechanical construction

The chosen method of text plotting is a Cartesian gantry system with two independent axes of movement and a pen up/down mechanism, consistent with the Cartesian arrangement illustrated by Teikari [3, p. 6, Fig. 4]. Complex kinematic equations and joint-dependent position errors are avoided by using separate x and y movements for reproduction. Linear GT2 timing belts and pulleys are used to convert rotary motion from stepper motors and axles to the desired Cartesian

movements. Timing belts were chosen for their cost effectiveness and ability to provide relatively fast travel without a lead-screw transmission.

A Cartesian gantry consists of stationary rails that guide movement in one axis, and a movable rail that guides the other axis of movement. For this system and the explanations that follow, the stationary rails guide *y*-axis movement, while the *x*-axis rail moves along the *y*-axis. Figure 3 shows the axes of the gantry. [2, hardware clarifications]

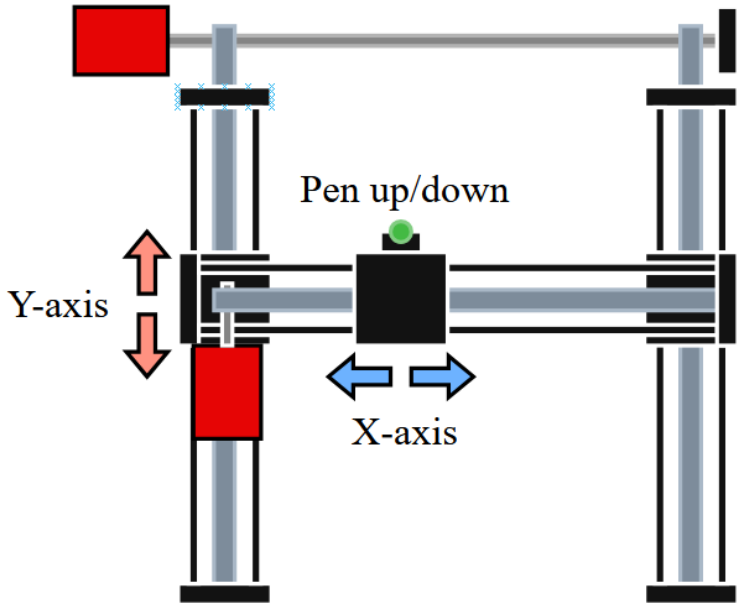


Figure 3. Cartesian gantry axes and pen up/down mechanism. [2, 1]

The *x*-axis rail needs to move with stability and consistency, which is why it is supported at both ends by the two *y*-axis rails. A single stepper motor was chosen to drive the pulleys on both sides to ensure synchronised motion and save costs. Two independent stepper motors driving one axis introduce a synchronisation concern because differences in missed steps or loading could offset one side relative to the other. The challenge with using one motor to drive two timing pulleys is extending its shaft to both pulleys without excessive load or twist in the new axle. A rigid coupler was used to extend the stepper motor shaft with a chromed steel rod. [2, 1, hardware clarifications; reproduce.py]

To reduce strain on the motors and allow smoother movement, linear bearings were fitted around the chromed rod rails so that the moving parts slide with low resistance. Bearing alignment is important as well as bearing diameter: an incorrectly aligned bearing can constrain the same rod in a different direction from the neighbouring bearing, increasing resistance instead of reducing it. During assembly, bearing fit and printed-part alignment were checked by sliding the parts along the rods before connecting the belts. The fit was adjusted where the assembled bearings restricted movement. [2, 1, hardware clarifications; reproduce.py]

The shared shaft couples the two sides mechanically, but does not remove all sources of error. Unequal belt tension, shaft torsion and a frame that is not square can still change the pen position. Belt tension must therefore be sufficient to maintain engagement without adding unnecessary bearing load. The pen holder also needs enough stiffness to prevent lateral deflection as the

direction of travel changes. These effects concern physical accuracy and cannot be inferred from the nominal step resolution alone. [2, 1, hardware clarifications; reproduce.py]

The printed components locate bearings, support the motors and pulleys, fasten the belts, and guide the pen mechanism. Their relationship within the completed assembly is identified in Figure 4. The arrangement fixes the relative positions of the rail, belt and pen, so manufacturing fit and bracket stiffness affect the same position budget as the drive. The available travel is established in Section 3.3.4. [2, hardware clarifications]

Figure requested: Photograph or assembled CAD view with the custom brackets, cross-shaft, belt tensioning, rail bearings and paper restraint labelled.

`design-gantry-assembly.jpg`

Figure 4. Mechanical assembly and the components locating the pen relative to the page. [2, 1]

3.3.2 Stepper motor selection and drive resolution

A stepper motor turns through fixed angular increments as the winding-current pattern changes. The windings are the motor's coils, and each current pattern establishes a magnetic field that pulls the rotor towards a corresponding position. The A4988 regulates these winding currents by chopping: it switches the drive to maintain the requested current level. Chopping continues even while a position is held; advancing the requested current pattern, rather than chopping itself, produces the next step [4, p. 9].

Two stepper motors are used in the gantry plotter, one for the x and one for the y direction. The selected motor is the 42BYGH40-01 NEMA 17 bipolar stepper, with a 40 mm body length, a 1.8° full-step angle and a rated current of 1.68 A per phase. Stepper control suits the required position commands because displacement follows from the number of commanded steps, without requiring an encoder for normal operation. The corresponding limitation is that the controller cannot determine whether every commanded step was physically completed. [2, 1, hardware clarifications; reproduce.py]

For the 20-tooth GT2 pulleys and 2 mm belt pitch, the travel per revolution and pitch radius follow from Equation 1. The radius is determined by tooth pitch, rather than by measuring the shaft or outside pulley diameter. [2, hardware clarifications]

$$\ell_{\mathrm{rev}} = N_t p = 20(2) = 40 \text{ mm/rev}, \qquad r_p = \frac{N_t p}{2\pi} = 6.37 \text{ mm}. \tag{1}$$

Here N_t is the tooth count and p is belt pitch. With 200 full steps per revolution and a microstep subdivision $\mu_s = 16$, the nominal commanded displacement is given by Equation 2. [2, hardware clarifications]

$$\kappa_0 = \frac{200\mu_s}{\ell_{\mathrm{rev}}} = 80 \text{ microsteps/mm}, \qquad \Delta x_0 = \frac{1}{\kappa_0} = 0.0125 \text{ mm}. \tag{2}$$

The A4988 supports this subdivision through its microstep-selection inputs [4, p. 6, Table 1]. The value in Equation 2 is substantially smaller than the proposal's 0.5 mm positional-error limit. It establishes adequate command granularity, but is not a measurement of accuracy: belt compliance, frame geometry, switch behaviour and pen deflection remain between the step count

and the mark on paper. Microstepping also does not guarantee that every intermediate mechanical position is equally accurate under load. [2, 1, hardware clarifications; reproduce.py]

Motor torque must be assessed at operating speed, rather than from holding torque alone. With feed F in mm/min, Equation 3 gives the motor speed. A travel feed of 4000 mm/min corresponds to 100 rev/min. [1, reproduce.py, GantryConfig]

$$n = \frac{F}{\ell_{\text{rev}}} = \frac{4000}{40} = 100 \text{ rev/min}. \tag{3}$$

The load model in Equation 4 separates translation and rotation. It applies to the belt-driven axis, with all lengths expressed in metres. [2, hardware clarifications]

$$T_{\text{req}} = r_p(F_f + ma) + J_{\text{rot}}\frac{a}{r_p}. \tag{4}$$

Calculated example: For a chosen acceleration case, $r_p = 0.006366$ m, $m = 0.950$ kg, $a = 0.5$ m/s^2, $F_f = 0.0373$ N and $J_{\text{rot}} = 10^{-5}$ kg m^2 give $T_{\text{req}} = 0.006366(0.0373 + 0.475) + 10^{-5}(0.5/0.006366) = 0.00405$ N m. The resistance and inertia are assumed; this is not the installed motor's verified torque margin. [2, hardware clarifications]

Here F_f is the resisting linear force, m is the translating mass, a is linear acceleration, and J_{rot} is the combined inertia of rotating parts referred to the motor shaft. The second axis carries the first-axis assembly, so its load must include that assembly. The rotational term must not also be included in m, which would count the same inertia twice. Equation 4 follows from $T = r_pF$ and angular acceleration a/r_p; it is a load calculation, not a substitute for the motor's measured torque–speed characteristic [5, page to verify] [*verify source/page:* `acarnley2002`]. [2, 1, hardware clarifications; reproduce.py]

At constant speed, only the resistance term remains. For the recorded moving mass of 0.950 kg, assuming $F_f = \mu_f mg$ with $\mu_f = 0.004$ gives approximately 0.000237 N m. This illustrative value indicates a small resistance load, but the coefficient is unmeasured and the calculation excludes acceleration. The driver starts each segment without a controlled acceleration ramp, so allowable starting rate also depends on motor current and reflected inertia. Establishing a dynamic margin requires those quantities and a matching motor characteristic, rather than the steady-state estimate alone. [2, 1, hardware clarifications; reproduce.py]

To complete: Confirm the 0.950 kg mass and which axis it represents; measure carriage resistance and obtain the torque–speed or starting-rate data for the installed 42BYGH40-01 at the actual driver setting. A curve for a different motor cannot establish its torque margin.

The ODROID coordinates the two axes using an integer error accumulator. For a move of a_x and a_y microsteps with $a_x \geq a_y$, the x axis is stepped on each iteration. The accumulator is reduced by a_y and a y step is inserted whenever it crosses zero, before adding a_x back. This is the Bresenham interpolation principle [6, p. 28]; it preserves the commanded endpoint and distributes the minor-axis steps along the line instead of completing one axis first. [2, 1, hardware clarifications; reproduce.py]

Algorithm 1 gives the major-axis form of the implemented loop. For a short illustrative command of eight x steps and three y steps, the initial error is four; y pulses occur on iterations 2, 5 and 7. The commanded endpoint is therefore $(8,3)$, with the minor-axis movement spread along the segment. Reversing a direction changes its sign, not the interpolation rule. [2, hardware clarifications]

Algorithm 1 Coordinated axis stepping with interruption checks [2, 1]

Require: Signed axis counts; pulse delay; connected GPIO interface.

```
(N_M, N_m) ← major and minor absolute counts; set directions
e ← ⌊N_M/2⌋
for i ← 1 to N_M do
    if abort requested or end stop confirmed then
        return interrupted, current position counters
    end if
    Pulse major axis; update its signed counter
    e ← e − N_m
    if e < 0 then
        Pulse minor axis; update its signed counter; e ← e + N_M
    end if
end for
return completed, current position counters
```

The geometry and timing of this loop must be considered separately. The current implementation sleeps inside each axis-pulse call, so an iteration containing two pulses takes longer than one containing a single pulse. For major- and minor-axis counts N_M and N_m, Equation 5 gives the delay contribution, excluding GPIO and interpreter overhead.

$$t_{\text{delay}} = (N_M + N_m)\,\tau, \qquad \tau = \max\left(\frac{d}{vN_M}, 2\times 10^{-6}\right), \quad v = F/60. \tag{5}$$

Calculated example: For an illustrative 10 mm move at $F = 800$ mm/min with $N_M = 800$, $N_m = 0$, $\tau = 10/[(800/60)800] = 0.0009375$ s and $t_{\text{delay}} = 0.750$ s. A 10 mm diagonal with $N_M = N_m = 566$ instead gives approximately 1.50 s of pulse delay. [1, reproduce.py, BresenhamMove]

Here d is move length in millimetres and v is the requested speed in mm/s. Equation 5 means that a 45° move has approximately twice the intended delay contribution, whereas an axis-parallel move does not. A shared pulse scheduler could reduce this directional speed variation. The present arrangement maintains simple coordinated positioning, but its feed value must not be treated as a measured trajectory speed. [2, 1, hardware clarifications; reproduce.py]

3.3.3 Pen lift mechanism and drive circuit

The pen has two required states: in contact with the page for a stroke, and clear of the page for travel. A compact geared motor and cam provide these states without adding a third positioning axis. The motor turns in one direction and the cam alternates the pen height as it rotates. A switch indicates the raised state. Compared with a servo, this arrangement uses a simple switched output and position indication, but it gives no commanded intermediate pen height or force. [2, hardware clarifications][1, reproduce.py]

Figure 5 shows the two pen positions. The holder and page support are part of the same design decision: repeatable motor movement is of little value if a flexible holder changes the nib position or if a stack of paper changes contact pressure. A single restrained sheet on a rigid support reduces this variation. The mechanism was adjusted experimentally to achieve clearance during travel without excessive pressure during drawing. [2, hardware clarifications]

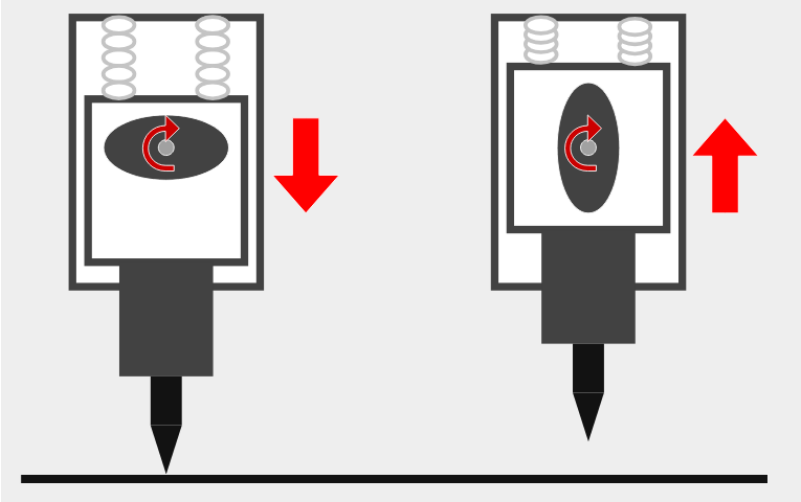


Figure 5. Cam-operated mechanism with the pen lowered and raised. [2, 1]

The software checks the switch before energising the motor. If the desired state is already present, no pulse is required; an unnecessary rotation could move the pen away from the requested state. Otherwise the motor runs until the switch indicates the target or the timeout expires. The current control routine allows a 40 ms overrun after confirmation of the raised state to seat the cam. This feedback operation differs from the G-code generator's settling dwell, which is described in Section 3.6.8. [2, 1, hardware clarifications; reproduce.py]

The low-side BC337 circuit in Figure 6 keeps the motor current out of the GPIO signal path. With a 1 kΩ base resistor and the assumed $V_{BE} = 0.7$ V, Equation 6 gives the approximate base current using the standard low-side resistor relation [7, section "Why NPN Devices Are Used in Low-Side Switches"].

$$I_B = \frac{V_{\text{GPIO}} - V_{BE}}{R_B} = \frac{3.3 - 0.7}{1000} = 2.6 \text{ mA}. \tag{6}$$

The calculation in Equation 6 establishes the nominal signal current only. It does not prove transistor saturation at motor startup or stall. The installed motor current, GPIO capability and transistor saturation conditions must be considered together [8, page to verify] [*verify source/page:* `bc337ds`]. Increasing base current without checking the GPIO rating would transfer the problem to the computer interface.

The installed pen motor is powered from the ODROID's dedicated 3.3 V supply pin, while the GPIO controls the transistor base through its resistor. The power pin and signal pin have different roles; moving the supply connection away from GPIO does not by itself establish that the rail can supply motor startup current. No flyback diode is fitted. Interrupting an inductive motor current can therefore impose a voltage transient on the switching stage [7, section "Protection and Wiring Techniques"]. A diode across the motor, reverse-biased during normal operation, would provide a recirculation path; its current and voltage ratings must be selected from the motor and supply conditions. This is a limitation of the installed circuit, not a protection feature already implemented.

To complete: Measure pen-motor running/startup current and confirm the 3.3 V rail's available capacity. The user reports 10 kOhm pull-downs; confirm whether one is fitted at the transistor base as well as at switch inputs. Replace the prototype image with correct DC notation and actual installed components. Record the reason the motor-suppression diode was omitted before final submission.

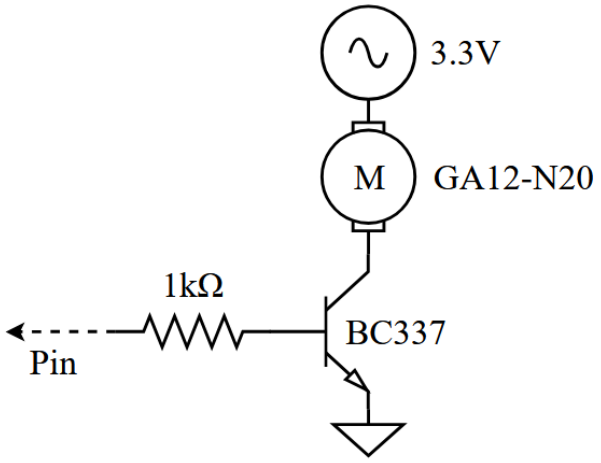


Figure 6. Prototype pen-motor switching circuit supplied with the draft. Supply and protection details require confirmation against the installed circuit. [2, 1]

3.3.4 End stops, homing and dimensional calibration

Four switches define the minimum and maximum travel of the two axes. Each is connected as an active-high input with a 10 kΩ pull-down, as shown in Figure 7. The pull-down defines the input while the contact is open. With the contact closed, its current is $3.3/10000 = 0.33$ mA and dissipation is approximately 1.1 mW. The resistor is between input and ground, so it does not limit fault current if the GPIO is incorrectly configured as an output driven low.

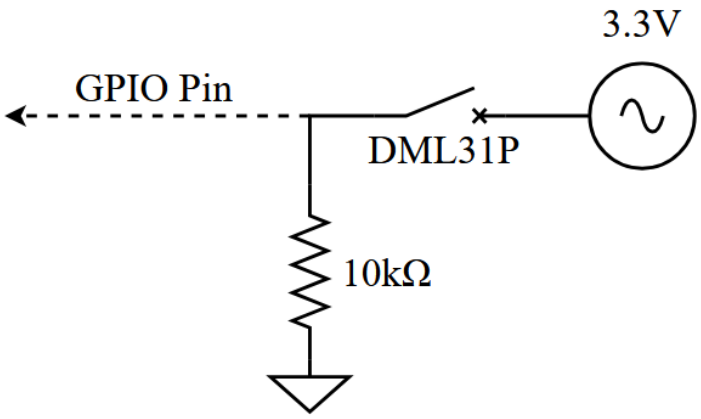


Figure 7. Active-high switch input with a 10 kΩ pull-down. [2, 1]

The control routine confirms a high reading with two further samples separated by 6 ms. Requiring three consistent readings rejects brief transients, at the cost of waiting approximately 12 ms when a potential trip is checked. Axis pulses are not continuously generated during this blocking check, so multiplying the full confirmation interval by the commanded feed would not correctly describe the software's movement during debounce. Physical stopping distance also depends on

the mechanics; it requires a measurement rather than a software-delay calculation alone. [2, 1, hardware clarifications; reproduce.py]

After startup, the step counters do not establish a known physical origin. Calibration therefore drives one axis towards either switch, reverses towards the other, and counts the switch-to-switch steps. Equation 7 converts this count to the scale used for motion. [1, reproduce.py, calibration geometry]

$$\kappa_x = \frac{N_x}{L_x}, \qquad \kappa_y = \frac{N_y}{L_y}, \qquad n_x = \mathrm{round}\big[(x+e)\,\kappa_x\big]. \tag{7}$$

Calculated example: Using the configured $L_x = 200.6$ mm and an illustrative counted $N_x = 16048$ gives $\kappa_x = 80$ microsteps/mm. A target $x = 20$ mm with $e = 5$ mm becomes $n_x = \mathrm{round}(25 \times 80) = 2000$. The step count is a chosen example, not a recovered calibration measurement. [1, reproduce.py, Calibrate]

Here N_x, N_y are measured step counts, L_x, L_y are the independently measured switch-to-switch travels, and $e = 5$ mm is the edge clearance. The corresponding expression applies to y. An axis retracts from the switch after calibration, and the origin lies inside the physical boundary. Absolute target counts are rounded and differenced against the tracked count, so rounding of one segment is not repeatedly accumulated as a new relative position error. [2, 1, hardware clarifications; reproduce.py]

The recorded travel constants in the current code are $L_x = 200.6$ mm and $L_y = 261.3$ mm. With 5 mm clearance on both ends, the software travel limits are 190.6 by 251.3 mm before the writing margins are applied. These are configuration values describing the available travel; a separate calibration record is needed to state the measured microsteps per millimetre. The result of calibration is not retained as a valid position across a restart. [2, 1, hardware clarifications; reproduce.py]

Calibration compensates for scale error, but it cannot correct an imprecise reference measurement. If the travel measurement has error δL, its first-order contribution to position error at commanded distance x is given by Equation 8.

$$|\delta x| \simeq |x| \frac{|\delta L|}{L}. \tag{8}$$

For example, allocating 0.25 mm to scale error over 190.6 mm of x travel requires the 200.6 mm reference travel to be known to approximately 0.263 mm. This allocation in Equation 8 is a design budget, not a measured uncertainty. Switch repeatability, backlash, pen compliance and squareness require the remaining error allowance. [2, 1, hardware clarifications; reproduce.py]

3.3.5 Electrical integration and power distribution

The motor-driver supply is operated at 24 V. The recorded supply-current indication is 0.35 A, with the supply current limit set to 2.11 A. These quantities have different meanings: the first is a reading at an operating condition, the second limits the available supply current, and neither specifies the winding-current regulation of an A4988. At the recorded operating condition, that supply delivers approximately $24(0.35) = 8.4$ W. This reading does not include a verified peak-load test or establish total consumption from both supplies. [2, hardware clarifications][1, reproduce.py]

The A4988 current-trip setting follows Equation 9 [4, p. 9].

$$I_{\text{trip,max}} = \frac{V_{\text{ref}}}{8R_S}. \tag{9}$$

Here R_S is the carrier's sense resistance and V_{ref} is the measured reference voltage. For illustration only, $R_S = 0.10\,\Omega$ and a 1.00 A trip setting would require $V_{\text{ref}} = 0.80$ V. The actual setting depends on the installed resistor and reference voltage. Available torque and driver heating depend on that setting; the motor's 1.68 A phase rating does not mean the driver is operating at 1.68 A. [2, 1, hardware clarifications; reproduce.py]

The reported resistor marking is tentatively R10, corresponding to 0.10 Ω if confirmed. This supports the illustrative resistance in Equation 9, but does not supply the missing measured reference voltage. The ODROID uses its supplied mains adapter; its DC output rating remains to be confirmed separately from the 24 V stepper supply. The dedicated ODROID 3.3 V rail supplies the installed 3.3 V loads, including the pen motor. The GPIO connections carry the associated control and feedback signals. [2, 1, hardware clarifications; reproduce.py]

The complete wiring needs a defined common logic reference between the ODROID, drivers and pen circuit. Motor-current returns should not be confused with signal paths, and driver supply decoupling must be located at the driver rather than relying on long supply leads. Figure 8 shows the complete supply and signal connections, with component values and the common reference identified. [2, hardware clarifications]

Figure requested: Complete circuit with GPIO offsets/physical pins, 24 V motor rail, ODROID supply, pen supply, driver logic rails, switch resistors and protection components.

`design-complete-circuit.pdf`

Figure 8. Complete electrical circuit and power distribution. [2, 1]

To complete: Confirm each driver's sense-resistor marking, measured Vref and cooling arrangement; record the ODROID adapter's DC output rating, the operating state during the 0.35 A reading, and fitted decoupling.

3.3.6 Image capture arrangement

The image-processing input depends on the camera arrangement as well as the algorithms. A camera looking approximately normal to a flat page reduces perspective distortion, and a restrained page reduces local curvature that a single deskew angle cannot remove. The capture area must include the full writing region while keeping the relevant strokes large enough to survive the line crop and network resize. The proposal specifies well-lit operation; software illumination normalisation is intended to accommodate residual variation within that arrangement. [2, hardware clarifications][1, reproduce.py]

The USB capture path returns a saved image to the same classification routine used for uploads. This gives both inputs a common orientation and segmentation procedure. The installed camera position, page restraint and illumination arrangement are identified in Figure 9. [2, 1, hardware clarifications; reproduce.py]

The camera is specified as an 8 MP USB autofocus unit with an IMX179 CMOS sensor and a 78° field of view. Its supplied resolution specification is 3264 by 2488 pixels. The capture

module opens the UVC device through OpenCV, reuses an active preview handle when available, and otherwise discards an initial sequence of five frames before saving the last readable frame. This gives the camera time to settle, but does not verify sharpness or autofocus convergence. The current routine does not explicitly request that advertised resolution, so the saved frame dimensions must be checked rather than equating sensor specification with operating resolution. [2, hardware clarifications]

Figure requested: Photograph of the actual camera and page mount, labelled with working distance, page area and illumination direction.

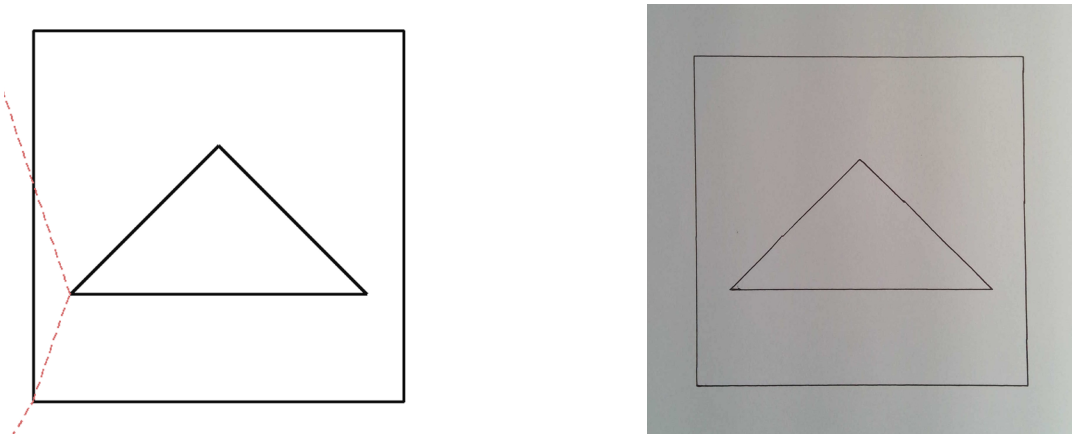
`design-camera-arrangement.jpg`

Figure 9. Page capture arrangement used to supply images to the processing pipeline. [2, 1]

To complete: Verify the supplied camera specification against its label and saved frame dimensions; add working distance and mounting dimensions. Describe only the lighting arrangement actually used; do not claim a measured lux range without a measurement.

3.3.7 Hardware verification and practical limitations

The purpose of hardware verification is to establish that a coordinate command produces the intended type of movement before handwriting is introduced. Axis-aligned lines expose scale and direction errors; diagonal lines expose coordinated stepping and frame geometry; repeated pen changes expose clearance and pressure problems. The square-and-triangle comparison already included in the draft is useful for this purpose (Figure 10). Its visual agreement supports correct interpretation of the commanded shape, but does not by itself establish the proposal's absolute accuracy or repeatability specification.



(a) Commanded trajectory (b) Physical drawing

Figure 10. Geometric commissioning example comparing the commanded path with the physical drawing. Formal positional qualification is reported in Results. [2, 1]

The design's principal strength is the direct relationship between orthogonal coordinates and motor displacement, combined with a mechanically shared drive on the two sides of the moving rail. Its principal uncertainties lie in physical compliance and open-loop execution. A missed step changes the actual position without changing the tracked count, and calibration cannot

retrospectively recover where that happened. Controlled acceleration, a more consistent pulse scheduler and improved mechanical constraint are possible remedies, but their expected benefit must be separated from the performance of the implemented machine. The formal dimensional and repeated-drawing measurements in Results assess the resulting mechanical accuracy. [2, 1, hardware clarifications; reproduce.py]

3.4 Page conditioning and line segmentation

The recogniser requires an image containing one line of text. A photograph, however, may contain uneven illumination, a rotated page, printed rules, page edges and strokes extending between neighbouring lines. The purpose of this stage is to remove those disturbances while preserving the shape and relative spacing of the handwriting. The implemented sequence in `segment.py` works mainly on a half-resolution image, then returns to the full-resolution image to build the line crops. [1, segment.py]

3.4.1 Input conditions and processing sequence

The working image is a two-dimensional array of grey values. Where a colour array reaches the deskew routine, its channels are averaged to obtain brightness. Reducing the analysis image by averaging each 2×2 block reduces its pixel count by a factor of four. Averaging is preferred to selecting every second pixel because a thin stroke then contributes to the reduced image even if it falls between sampling positions. A further reduced mask is sufficient for the angle search, where only the arrangement of whole lines is needed. [1, segment.py]

The intermediate representations in Figure 11 separate illumination correction, ink extraction, line boundaries and final crops. Inspecting these representations localises a missing stroke to the operation that removed it. The output includes the line images together with their order and diagnostic information such as skew angle, pitch and rejected crops. [1, segment.py]

A straight horizontal boundary is adequate only when there is a blank row between lines. Handwritten descenders and ascenders can overlap in vertical position without belonging to the same line. The chosen design first locates the lines, finds curved boundaries between them, then uses component geometry to refine which line owns each ink piece. This is more appropriate than treating every connected ink component as a character, since cursive words may form a single component. [1, segment.py]

Algorithm 2 links the representations used in Figure 11. The order matters: rules and artefacts are removed before estimating final line centres, whereas faint-ink recovery occurs after ownership has been established. [1, segment.py]

3.4.2 Lighting normalisation and ink extraction

The first requirement is to distinguish ink from paper under an illumination gradient. A single threshold applied directly to brightness can classify a shadowed part of the page as ink while

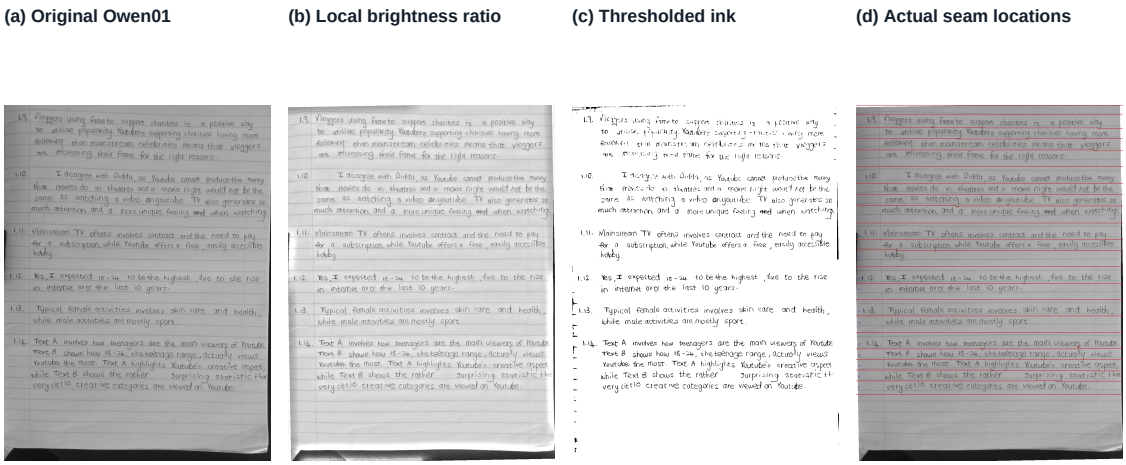


Figure 11. Processing stages used to obtain separate text-line images from a photographed page. [2, 1]

Algorithm 2 Photographed page to ordered line crops [2, 1]

Require: Grey page on its chosen line axis.
Ensure: Ordered full-resolution crops, coordinates and diagnostics.

$(G_f, G_h, M, R, \theta) \leftarrow \text{Deskew}(G)$
$M \leftarrow \text{RemovePrintedStructure}(M, G_h)$
$(M, \mathcal{C}) \leftarrow \text{DropArtifactPieces}(M, R)$
$(P, c) \leftarrow \text{LineCentres}(M)$
$b \leftarrow \text{SeamCuts}(M, c, P)$
$O \leftarrow \text{AssignOwnership}(M, \mathcal{C}, b, c, P)$
$\mathcal{L} \leftarrow \varnothing$; normalise full-resolution brightness
for each line index n **do**
  $C_n \leftarrow \text{BuildLineCrop}(n, O, G_f, P)$
  if C_n has sufficient ink and height **then**
    Append C_n, its fixed-canvas view and bounding box to $\mathcal{L}$
  end if
end for
$\mathcal{L} \leftarrow \text{DropSolidCrops}(\mathcal{L})$; renumber retained crops
return $\mathcal{L}$, seams, ratio image and diagnostics

losing pale strokes elsewhere. Equation 10 compares each pixel with its own local background estimate instead.

$$R(y,x) = \frac{G(y,x)}{\max[B_r(y,x),1]}, \qquad M(y,x) = \mathbf{1}[R(y,x) < t^*]. \tag{10}$$

Calculated example: With illustrative $G = 90$, $B_r = 180$ and $t^* = 0.70$, $R = 90/180 = 0.50$ and $M = 1$. Paper at $G = 171$ under the same background gives $R = 0.95$ and $M = 0$. [1, segment.py, Deskew]

Here G is grey intensity on the 0–255 scale, B_r is a square-window mean with radius r, R is normalised brightness, and M is the binary ink mask. The floor of 1 prevents division by zero in black regions. The code uses $r = \max(8, \min(40, \text{round}(H/41)))$, where H is the working-image height. The radius follows image size within bounded limits: it must average over strokes without following their darkness as if it were illumination. [1, segment.py]

The mean is evaluated through a summed-area table [9, page to verify] [*verify source/page:* `crow1984`]. For an exclusive prefix sum S, the window sum is obtained from four accesses, as shown in Equation 11.

$$\Sigma_{[y_0,y_1)\times[x_0,x_1)} = S(y_1,x_1) - S(y_0,x_1) - S(y_1,x_0) + S(y_0,x_0). \tag{11}$$

Calculated example: For four prefix entries $S_{11} = 900$, $S_{01} = 300$, $S_{10} = 200$ and $S_{00} = 80$, the window sum is $900 - 300 - 200 + 80 = 480$. A four-pixel window therefore has $B_r = 120$. [1, segment.py, Integral and BoxSum]

The window area divides this sum to give B_r. Equation 11 makes the cost per window independent of its radius after the prefix image has been constructed. Edge replication is used by this local-mean routine so that the boundary is not darkened by artificial zero padding. [1, segment.py]

Otsu's criterion selects t^* from the ratio histogram rather than from the original grey histogram [10, page to verify] [*verify source/page:* `otsu1979`]. Equation 12 gives the between-class variance being maximised.

$$t^* = \arg\max_t \ \omega_0(t)\omega_1(t)\big[\mu_0(t) - \mu_1(t)\big]^2. \tag{12}$$

Calculated example: For a constructed candidate split with $\omega_0 = 0.2$, $\omega_1 = 0.8$, $\mu_0 = 0.3$ and $\mu_1 = 1.0$, the criterion is $0.2(0.8)(0.3 - 1.0)^2 = 0.0784$. A candidate giving 0.060 is inferior; the implementation searches all valid histogram splits. [1, segment.py, Otsu]

Here ω_0, ω_1 are the histogram masses below and above a candidate threshold and μ_0, μ_1 are their means. The normalisation in Equation 10 first brings paper towards a common level, allowing the global criterion in Equation 12 to operate on a more consistent image. The implementation uses 256 bins over the clipped ratio range 0–1.2. [1, segment.py]

Large solid regions are then removed because the dark surround or a binder interior can survive thresholding without representing handwriting. The page region is estimated from its bright connected area, and long nearly horizontal or vertical ink runs are masked as printed structure. Run thresholds are expressed relative to line pitch, rather than fixed photograph dimensions. The horizontal and vertical limits are respectively 1.25 and 1.4 pitches. These are geometric heuristics: a long handwritten underline can satisfy the same condition as a printed rule, so the algorithm is designed for text extraction rather than preserving every mark on a page. [1, segment.py]

3.4.3 Page orientation and skew correction

Orientation is treated at two scales. A quarter turn changes which axis contains the line gaps; a small skew changes how sharply ink is concentrated along rows. For the quarter-turn decision, the code compares the fraction of nearly empty rows in the current image and its 90° rotation. A turn is accepted only when the rotated score exceeds the original by a factor of 1.35. This margin avoids rotating a page merely because two weak scores differ slightly. Sparse or unusually arranged text can still make the row-gap evidence ambiguous. [1, segment.py]

After choosing the line axis, the deskew routine tests angles from −4° to +4° at 0.5° spacing. Equation 13 gives its objective, related to projection-profile methods [11, page to verify] [*verify source/page:* `postl1986`].

$$\theta^* = \arg\max_{\theta} \sum_{y} p_\theta(y)^2, \qquad p_\theta(y) = \sum_{x} M_\theta(y,x). \tag{13}$$

Calculated example: Two candidate row profiles $(0,6,0,6)$ and $(3,3,3,3)$ contain the same twelve ink pixels, but score 72 and 36. The first is preferred because the ink is more concentrated in rows. [1, segment.py, Deskew]

Here p_θ is the row ink count at a candidate angle. Squaring favours concentrated rows over the same amount of ink spread across many rows. Rather than rotating the complete mask for every trial, the implementation maps ink coordinates to candidate rows and counts them. The selected rotation is then applied to the full and working grey images, and the mask is recomputed. This avoids repeatedly interpolating a binary image. [1, segment.py]

A page and its upside-down version have the same line-axis structure. The server therefore uses the reader to compare up to three sampled line crops in both directions. It compares the mean best-frame log score, reuses the selected readings and reverses line order when necessary. This is an orientation heuristic, not an independent measure of text correctness. A reader that is confidently wrong can select the wrong orientation, particularly when little text is available. [1, segment.py]

3.4.4 Line detection and separation

The line pitch P provides the main scale reference for later decisions. A smoothed row profile is formed from the central writing region; candidate peaks must exceed 6% of its maximum. The preferred estimate is the median distance between peaks, subject to a lower bound based on component height. This check prevents the ascender, body and descender bands of a single large line from being treated as separate line spacings. Autocorrelation and a component-height estimate provide fallbacks when direct gaps are unsuitable. [1, segment.py]

Line centres are selected in order of peak strength, with a minimum separation of $0.6P$. This suppresses several peaks within the same line. The code also supports merging peaks separated by a shallow valley, but that additional rule is not automatically active for every photographed page; its default threshold disables merging. The distinction matters because a rule that improves rendered text need not improve the shadows and artefacts of a real photograph. [1, segment.py]

Between each pair of centres, a minimum-cost seam crosses the image from left to right. The dynamic programme in Equation 14 follows the seam-carving principle [12, PDF p. 3, Sec. 3].

$$D(y,x) = C(y,x) + \min_{|d|\le s}\{D(y+d,x-1) + 0.02|d|\}. \tag{14}$$

Calculated example: For local cost $C = 0.10$ and predecessor pairs $(D,d) = (0.30,0),(0.20,1),(0.25,-1)$, the accumulated cost is $0.10 + \min(0.30,0.22,0.27) = 0.32$. The selected predecessor changes row by one. [1, segment.py, LeastCostSeam]

Here D is accumulated cost, C is the local cost and $s = 7$ is the allowed row change per column. Backtracking from the minimum final cost returns the boundary. Equation 14 penalises unnecessary vertical movement while allowing a boundary to bend around a descender. [1, segment.py]

The cost in Equation 15 strongly discourages crossing ink and weakly attracts the boundary to the space between line centres.

$$C(y,x) = 4\widetilde{M}(y,x) + 0.012\left(\frac{y-y_m}{h_g}\right)^2. \tag{15}$$

Calculated example: With $h_g = 20$ pixels, blank ink and $y - y_m = 10$ pixels, $C = 0.012(10/20)^2 = 0.003$. Crossing ink at the midpoint costs 4, over a thousand times as much locally. These are the implemented weights, not measured probabilities. [1, segment.py, SeamCuts]

Here $\widetilde{M}$ is the mask after a closing operation, y_m is the midpoint of the neighbouring centres and h_g is half their separation. The closing radius is $\max(2,\lfloor 0.07P \rfloor)$ pixels. Closing fills narrow gaps that would otherwise let a seam enter a stroke, without simply treating a thick dilation around every letter as real ink. In Equation 15, the midpoint term resolves low-ink alternatives; it is deliberately much weaker than the ink penalty. The examples in Section 3.4.6 examine the resulting boundaries and the cases that still defeat these geometric preferences. [1, segment.py]

A new sensitivity check on six fixed gaps from two real pages varies the midpoint and step penalties separately. At the implemented 0.012/0.02 setting, mean midpoint offset is 0.363 working pixels; removing the midpoint attraction raises it to 4.055 pixels. All tested settings still cross two raw-ink pixels across those seams, so this checks the weights' effects rather than proving an optimum. Appendix B.6 gives the settings and retained outcomes [2, `Evidence/coefficient_sensitivity.json`].

3.4.5 Ink ownership and classifier crop preparation

Seams initially divide the image into bands, but a pixel-by-pixel cut can separate a dot from its stem or clip a curved descender. Connected-component ownership therefore refines the band assignment. Components are found with row runs and union operations, following connected-component labelling principles [13, pp. 476–479]. The implementation includes diagonal contact, giving eight-connected components. Large pieces can still contain writing from more than one line, so component labelling alone is not a complete line assignment.

Algorithm 3 summarises the correction order. For a component spanning several bands, a band holding at least 15% of its in-band ink is substantial. The uppermost substantial band supplies the reference owner for small-piece corrections; otherwise the largest overlap is used. Large components are not indiscriminately moved into that band: their pixels retain seam-band assignment except where the small-piece and narrow-tail rules apply. This distinction preserves a word's main body while recovering a protruding part. [1, segment.py]

Algorithm 3 Refining seam bands into ink ownership [2, 1]

Require: Ink mask M, components $\mathcal{C}$, seams, centres and pitch P.

for each component $C \in \mathcal{C}$ **do**
  Count overlap $a_{C,n}$ with each seam band n
  $S \leftarrow \{n : a_{C,n} \geq 0.15 \sum_j a_{C,j}\}$
  Choose topmost S as reference owner, or maximum overlap if $S = \varnothing$
  Correct eligible narrow tails and small pieces; reject remote specks
  Associate compact dots with plausible stems using pitch-scaled guards
end for
$O \leftarrow$ corrected ownership, preserving large-piece seam assignments
for each raw component near a neighbouring line **do**
  if detached-dot or enclosed-ring guards pass **then**
    Transfer eligible pixels to the supported stem's line; protect correction
  end if
end for
Reassign residual fragments lacking their own body-band support
return O; excluded pixels retain no line index

An `i` dot between two text lines illustrates the need for this sequence. A seam may put it with the upper line because of its vertical position, while a stem below supplies evidence for the lower line. Closing used for component analysis can also join the dot to an unrelated descender; the raw mask is therefore revisited rather than treating the closed component as unquestionable evidence. A ring-shaped mark needs the additional hole test so that an open hook is not treated as a detached loop. These are geometric decisions with ambiguous cases, rather than knowledge of the intended character. [1, segment.py]

The final crop is built from the full-resolution image and the refined ownership map. Contrast adjustment improves faint ink, while ownership restricts recovery so that strengthening one line does not simply pull a neighbouring stroke into it. The recogniser receives an aspect-preserving fit on a 64 by 640 canvas. Equation 16 defines the common scale for an original crop of height h and width w.

$$a = \min\left(\frac{64}{h}, \frac{640}{w}\right), \qquad (h', w') \simeq (ah, aw). \tag{16}$$

Calculated example: A chosen 80×1000 crop gives $a = \min(0.8, 0.64) = 0.64$, hence an approximately 51×640 image plus white vertical padding. Its aspect ratio is retained, but its text body is shorter than 64 pixels. [1, segment.py, FitFixedCanvas]

Remaining space is padded white, with the ink left-aligned and vertically centred. The shared scale in Equation 16 avoids stretching narrow letters differently from wide ones. Very long lines, however, become short in height when fitted to the width limit, which reduces the detail available to the networks. [1, segment.py]

The full-resolution crop uses a soft interpolation of its own ink mask to avoid jagged cut edges. A separate ownership exclusion is resized with nearest-neighbour interpolation: interpolating a yes/no ownership decision could partly return a neighbour's ink. Local thresholding can recover faint strokes within a neighbourhood of this line, but cannot recover ink already assigned to another line. Contrast is stretched between a robust ink level and paper level, followed by a darkness exponent of 0.55. This strengthens faint writing without replacing its geometry with a solid rectangular band. [1, segment.py]

The final solid-crop guard rejects a crop only when its dark-pixel fraction exceeds both 15% and three times the page's median crop fraction. Requiring both prevents one unusually dense but genuine line from being rejected merely because other lines are sparse. This reduces the chance of a solid shadow receiving a strong writer vote in Section 3.7. If every crop would be rejected, the implementation retains the input crop list; crop rejection is therefore a guard rather than a guarantee of valid text. [1, segment.py]

3.4.6 Segmentation verification and limitations

Figure 12 uses three actual photographs of the same three-line handwritten page from the camera roll. Their shadows and page presentation differ; no illuminance or physical rotation angle was recorded. The unchanged native images were processed through the line-axis decision and segmentation, without manually correcting an unsuccessful orientation. [1, segment.py]

The striped-shadow capture produced three crops. The diffuse-shadow capture produced four, so matching the writing visually remains necessary even under milder-looking illumination. The third capture was left on the wrong line axis: its gap scores were 0.379 and 0.400, giving a ratio of only 1.054 against the required 1.35, and segmentation produced 28 slices. Reaching the $-4°$ deskew limit could not correct a missed quarter turn. These cases expose a sparse-page limitation of row-gap orientation; they do not establish a lighting operating range or a success rate. An acquisition guide or a separately verified orientation method would address the ambiguity before fine deskewing. [1, segment.py]

The fuller Owen page in Figure 11 produced 20 crops at native resolution, matching its 20 labelled text lines. That count checks gross splitting, not every boundary pixel. Appendix B records the capture filenames, extended digital perturbations and independent algorithm checks, including failures after additional image reduction. [1, segment.py]

The scale-based thresholds and curved boundaries accommodate changes in handwriting size and overlapping vertical extents without assigning every page a fixed pixel spacing. Their assumptions also define the limitations. Highly irregular line spacing weakens a single pitch estimate, touching strokes from two lines can defeat component ownership, and sparse pages provide weak orientation evidence. Severe perspective or page curvature is not corrected by a global rotation. Improving the mount addresses the latter at source; more general page rectification would add processing and require its own verification. [1, segment.py]

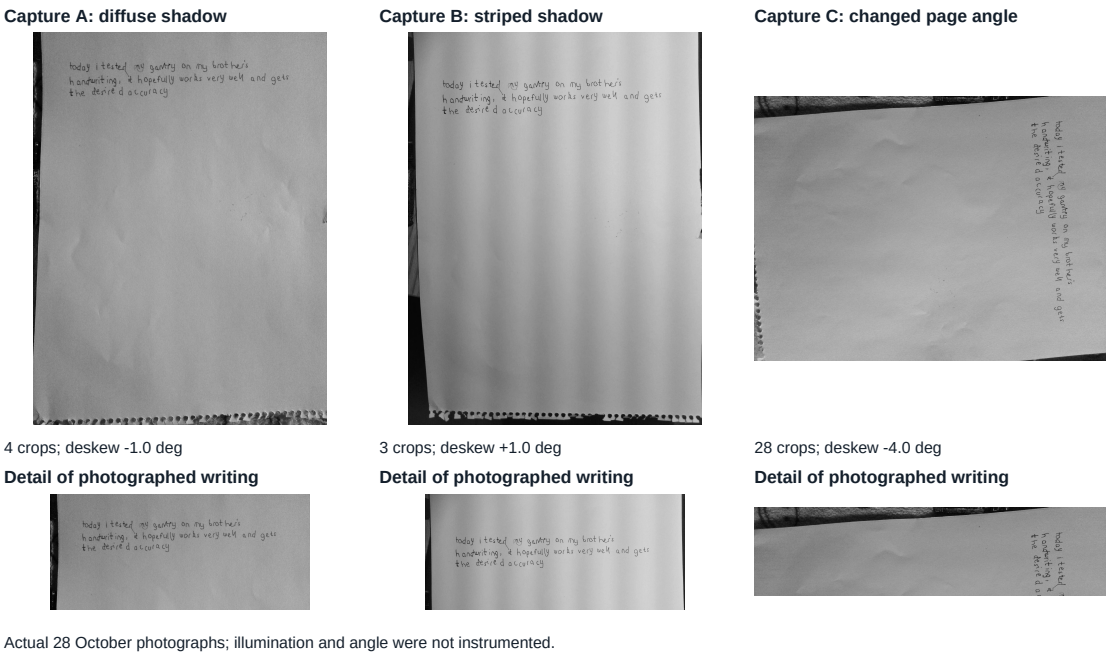


Figure 12. Actual repeated photographs and segmentation diagnostics for the same sparse page. Capture C retains the unsuccessful axis decision. [2, 1]

3.5 Neural network design, training and classification

The classification stage has two outputs: the text in a line and the identity of its writer. Both depend on stroke features, but text recognition must preserve their horizontal order while writer identification must combine evidence across the line. Two networks therefore use the same convolutional architecture with different heads. The recogniser follows the CNN–BiLSTM approach of Kizilirmak and Yanikoglu [14, pp. 4–5]; the project implements and integrates that architecture with its own preprocessing, training and handwriting-reproduction pipeline.

3.5.1 Training data, labels and evaluation separation

Public IAM handwriting [15, page to verify] [*verify source/page:* `marti2002`] and the photographed personal contributions provide examples with known text and writer identity. Public line images supply text-recognition training data, while writer-labelled pages and the personal samples support writer identification and style extraction. The eleven writers selected for the reported evaluation and intended demonstration are six database writers (150, 152, 153, 384, 552 and 588) and five personal contributors. A stored profile, a classifier output class and an evaluated writer are different objects; the checkpoint's recorded class mapping defines how its output indices are interpreted.

The data pipeline pairs every crop with its transcript and writer label. The same preparation functions are used by the first-principles and framework training routes so that a comparison does not silently change segmentation or label handling. Blank or unusable labelled lines are excluded.

Characters outside the configured alphabet are filtered by the label encoder; they cannot be recognised as separate output symbols unless the alphabet and output layer are extended. [1, recognise.py; train.py; trainlibrary.py]

The page-based writer dataset holds out an IAM page per writer, while the personal data use a fixed-seed 15% line split. A page split and a line split provide different separation: a held-out personal line can share camera conditions, page background and writing session with training lines. It is therefore appropriate to describe those samples as held-out lines, rather than as wholly unseen capture sessions. Augmented versions of a sample must remain within its assigned training partition. [1, recognise.py; train.py; trainlibrary.py]

Preparing public forms also requires excluding printed prompts and signatures before matching detected lines to their transcripts. The eleven-writer classifier corpus contains 1186 labelled lines: 517 IAM and 669 photographed lines, split into 1031 training and 155 held-out lines. The final reader instead uses a joint corpus of Teklia pre-cut IAM lines and the five photographed writers; its exact training and validation counts are not preserved in the available record. [2, 1, project records; implementation]

Evaluation separation concerns three inputs: the writer classifier's training lines, the reader's training lines and the pages used to build glyph profiles. Holding a line out of one classifier does not exclude it from the other corpus, and constructing a profile from an evaluation page already exposes its letterforms. Cross-corpus source identifiers and profile-page manifests are therefore needed before describing combined performance as independent of those inputs. [1, recognise.py; train.py; trainlibrary.py]

To complete: Record reader split counts and the source pages used to construct final glyph profiles. Keep the reported writer holdout distinct from a wholly independent capture-session test.

3.5.2 Shared convolutional feature extractor

The input tensor has dimensions $(B, 1, 64, 640)$, where B is batch size. The grey crop is inverted and scaled to $[-1, 1]$, making dark ink positive and white padding negative. Table 2 gives the implemented feature-map dimensions. Two pooling operations reduce the width to 160 positions; there is no third spatial pooling stage before height is collapsed.

Stage	Operation	Output excluding batch
Input	Aspect-preserving line fit and intensity scaling	$1 \times 64 \times 640$
1	Two 3×3 convolutions, each with batch normalisation and ReLU; 2×2 max pool	$32 \times 32 \times 320$
2	Four convolution/normalisation/ReLU blocks; 2×2 max pool	$64 \times 16 \times 160$
3	Six convolution/normalisation/ReLU blocks	$128 \times 16 \times 160$
Height reduction	Maximum over the remaining height	$128 \times 1 \times 160$

Table 2. Implemented convolutional backbone shared in architecture by the text and writer networks. The two models have separately trained parameters. [2, 1]

The layer operation in Equation 17 shows what the numerical implementation evaluates. For output channel o, the input-channel and kernel sums combine into one feature value.

$$z_{o,y,x} = b_o + \sum_c \sum_{i,j} W_{o,c,i,j} X_{c,y+i,x+j}. \tag{17}$$

Calculated example: In one illustrative single-channel 3×3 patch, set the nine inputs to $1, \ldots, 9$, all kernel weights to one and the bias to 0.5. The output is $z = 0.5 + 45 = 45.5$ before normalisation and ReLU. These are illustrative weights, not learned parameters. [1, recognise.py, Conv2d]

Here W is the learned kernel, b its bias and X the padded input. Equation 17 is the cross-correlation convention used by the implementation. Image patches are rearranged into a matrix and multiplied by flattened kernels, rather than evaluating a Python loop for every multiply. This follows the matrix formulation of convolution [16, page to verify] [*verify source/page:* `chellapilla2006`]. The choice improves the practicality of the numerical route while leaving the layer's calculation visible. [1, recognise.py; train.py; trainlibrary.py]

Batch normalisation evaluates Equation 18, with channel statistics computed over the batch and spatial dimensions during training [17, PDF pp. 4–5, Alg. 1].

$$\widehat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \widehat{z} + \beta. \tag{18}$$

Calculated example: For illustrative $z = 3$, $\mu_B = 1$, $\sigma_B^2 = 4$, $\varepsilon = 10^{-5}$, $\gamma = 1.5$ and $\beta = 0.1$, $\widehat{z} = 2/\sqrt{4.00001} \simeq 1$ and $y \simeq 1.6000$. [1, recognise.py, BatchNorm2d; train.py, BatchNorm2dForwardTrain]

Here γ, β are learned channel parameters and ε stabilises division. At inference, stored running statistics replace μ_B, σ_B^2, making Equation 18 a fixed affine operation. The following ReLU applies $\max(0, y)$ and max pooling retains the largest feature in each local window. These operations progressively convert pixel neighbourhoods into learned responses while reducing the spatial size passed to the task heads. [1, recognise.py; train.py; trainlibrary.py]

Figure 13 makes the different uses of horizontal position explicit. The term "shared" refers to architecture and numerical building blocks; it does not mean that the trained writer classifier uses the reader's current weight tensors. [1, recognise.py; train.py]

3.5.3 Text recognition using sequence modelling and CTC

The text head treats the 160 feature columns as a sequence. Each column contains 128 features after height pooling. Two bidirectional LSTM layers, each with 256 hidden units per direction, combine evidence from both sides of a character before a linear head produces scores over the alphabet and the blank class. The configured alphabet contains 73 characters, giving 74 output classes including blank. The chosen input geometry and decoding are part of this project's implementation rather than an assertion that every setting in the reference study was reproduced. [1, recognise.py; train.py; trainlibrary.py]

The LSTM state update in Equation 19 retains an additive memory path [18, p. 5, Eqs. (9)–(11)].

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t). \tag{19}$$

Calculated example: One scalar cell with $c_{t-1} = 0.5$, $f_t = 0.8$, $i_t = 0.3$, $g_t = 0.4$ and $o_t = 0.7$ gives $c_t = 0.8(0.5) + 0.3(0.4) = 0.52$ and $h_t = 0.7 \tanh(0.52) = 0.3344$. [1, recognise.py, LstmForward]

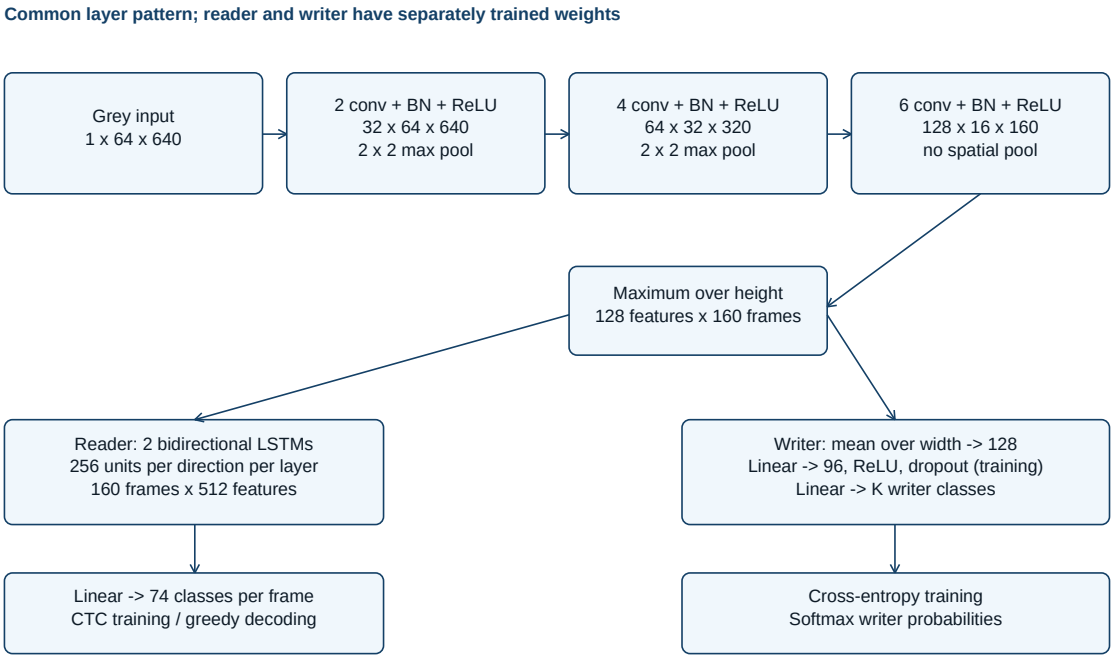


Figure 13. Architecture of the text reader and writer classifier. [2, 1]

Here c_t is cell state, h_t is hidden output, i_t, f_t, o_t are sigmoid input, forget and output gates, and g_t is the candidate update. Each is computed from the current feature vector and previous hidden state. The implementation omits the additional cell-to-gate peephole connections shown in the cited source. The backward direction applies the same type of recurrence to the reversed sequence, and the two outputs are concatenated. Equation 19 explains why the implemented reader has sequence state in addition to convolutional features. [1, recognise.py; train.py; trainlibrary.py]

The gates are obtained from one matrix calculation, split in input, forget, candidate and output order: $[a_i, a_f, a_g, a_o]^T = W_x x_t + W_h h_{t-1} + b_x + b_h$. Sigmoid is applied to a_i, a_f, a_o and tanh to a_g. Concatenating both directions gives 512 features per frame; the second recurrent layer therefore receives 512 inputs rather than the first layer's 128. At inference the entire line is available, so using right-hand context does not require predicting future input. [1, recognise.py; train.py; trainlibrary.py]

CTC trains from a line transcript without requiring character positions [19, PDF pp. 2–3]. Equation 20 sums the probabilities of frame paths that collapse to the target string.

$$\mathcal{L}_{\mathrm{CTC}} = -\log \sum_{\pi:\mathcal{B}(\pi)=\ell} \prod_{t=1}^{T} p_t(\pi_t). \tag{20}$$

Calculated example: For a two-frame, two-symbol example targeting the symbol a, let $p_1(a) = 0.8$ and $p_2(a) = 0.7$, with the remaining mass assigned to blank. The admissible paths $(a,a),(a,\varnothing),(\varnothing,a)$ sum to $0.56 + 0.24 + 0.14 = 0.94$, giving $\mathcal{L} = -\log(0.94) = 0.06188$. [1, train.py, CtcLoss]

Here $p_t(k)$ is the frame probability for symbol k, ℓ is the transcript, and $\mathcal{B}$ merges consecutive repeats before deleting blanks. For example, the path $a,a,\varnothing,b,b$ collapses to "ab", while $a,\varnothing,a$ preserves "aa". The dynamic programme uses blank-interleaved states with transitions that stay,

advance one state, or skip a blank when the adjacent labels differ. Arithmetic is performed in log space to avoid multiplying many small probabilities. [1, recognise.py; train.py; trainlibrary.py]

The implementation produces $T = 160$ frames. A target with L labels and r adjacent repeated-label pairs requires at least $L + r$ frames. The extended lattice has $2L + 1$ states, but not every state must be visited; that number is not a universal minimum sequence length. At inference, greedy frame decisions are collapsed by the same rule. This avoids a dictionary or language model, but a locally plausible wrong frame path can remain wrong even where a beam search could recover an alternative. [1, recognise.py; train.py; trainlibrary.py]

Algorithm 6 describes the numerical loss and derivative. Let $\alpha_{t,s}$ include the current state's emission, while $\beta_{t,s}$ accounts only for emissions after frame t. The derivative used by the implementation is then Equation 21, with the occupancy convention obtained by adapting the CTC forward–backward derivation [19, PDF pp. 5–6].

$$q_{t,k} = \sum_{s:\ell'_s=k} \exp(\alpha_{t,s} + \beta_{t,s} - \log Z), \qquad \frac{\partial \mathcal{L}}{\partial z_{t,k}} = p_t(k) - q_{t,k}. \tag{21}$$

Calculated example: In that example, the posterior occupancy of a at frame 1 is $q_{1,a} = (0.56 + 0.24)/0.94 = 0.85106$. Its unaveraged logit derivative is $0.8 - 0.85106 = -0.05106$, encouraging more probability on the target symbol. [1, train.py, CtcLoss]

Here Z is the total probability of admissible target paths, z denotes logits and q is posterior class occupancy. The convention for β avoids counting the current emission twice. Equation 21 is averaged over the batch in the trainer, without an additional target-length division. Algorithm 6 in Appendix A.5 gives the full forward–backward procedure with explicit blank/repeat transitions. [1, recognise.py; train.py]

For the illustrative transcript "letter", the two adjacent `t`s require separate emission runs with a blank between them. Advancing directly from one `t` state to the next would collapse to a single character, so that skip transition is forbidden. This same restriction is retained during forced alignment in Section 3.6.2; training sums admissible paths, whereas extraction selects one. [1, recognise.py; train.py; trainlibrary.py]

3.5.4 Writer identification and stroke normalisation

Writer identification should combine the shape evidence distributed along a line. It therefore averages the 160 column features rather than passing them through a sequence decoder. A 128-element feature vector is mapped through a 96-unit hidden layer, ReLU and a final K-class output. Dropout with probability 0.2 is active in the hidden head during training and omitted during inference [20, pp. 1931, 1934]. Here K is the number of writers represented by that checkpoint.

The initial classifier operated on handwriting as photographed and performed well on those samples. However, such images contain more than letter shape: pen width, darkness and the capture process can also distinguish writers. The gantry uses its own pen, so reproducing the selected writer's geometry does not reproduce the same ink appearance. A classifier that relies on those cues would therefore provide an unsuitable basis for judging whether the robot preserved the writer's style. [1, recognise.py; train.py; trainlibrary.py]

The later classifier uses stroke-normalised input. Ink is binarised, reduced to a centreline with Zhang–Suen thinning [21, p. 237], then expanded to a controlled width before the fixed-canvas fit. Equation 22 describes the implemented transformation.

$$I_s = 255\left[1 - \delta_k\left(\text{thin}(M)\right)\right], \quad w = \text{clip}\left(\text{round}(\rho h_x), 2, 9\right), \quad k = 2\lfloor w/2 \rfloor + 1. \tag{22}$$

Calculated example: At the selected $\rho = 0.17$ and an illustrative estimated $h_x = 24$ pixels, $w = \text{clip}(\text{round}(4.08), 2, 9) = 4$ and the actual odd kernel is $k = 5$. The redraw is 0 at dilated ink pixels and 255 elsewhere. [1, recognise.py, StrokeNormalize]

Here h_x is estimated x-height, ρ is the selected width fraction and δ_k is square dilation with an odd k-pixel kernel. The actual discrete kernel is important: nearby fractions can produce exactly the same transformed image. The code contains a default fraction of 0.14 and supports explicit training widths; the final model and its preprocessing must use matching settings. [1, recognise.py; train.py; trainlibrary.py]

The stroke-normalised classifier is used for writer identification and synthesis evaluation in the final design described here. This reduces dependence on ink appearance while retaining the letter geometry that the gantry can reproduce. It does not make the classifier completely independent of capture conditions, since thresholding and skeletonisation can themselves change a faint or touching stroke. It also removes potentially useful evidence from original handwriting, making the choice a deliberate trade-off between photograph-specific discrimination and consistency across original and reproduced writing. [1, recognise.py; train.py; trainlibrary.py]

Figure 14 shows this transformation on existing real crops. The comparison makes the individual operations visible: binary extraction defines what ink is retained, thinning reduces it to a centreline, and dilation restores a controlled width. A matched ink-versus-shape comparison on original and reproduced samples can then test the reason for the choice without substituting classifier confidence for independent stylistic agreement. [1, recognise.py; train.py]

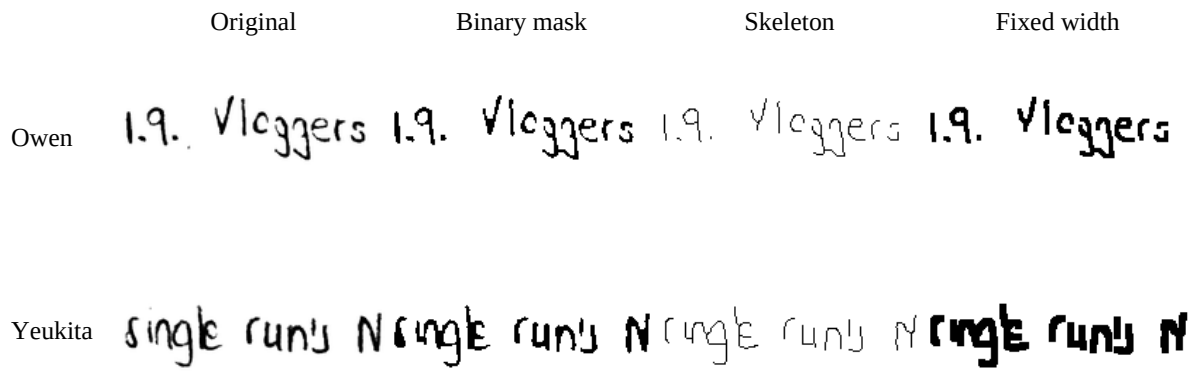


Figure 14. Stroke-normalisation stages on selected Owen and Yeukita crops at the selected width fraction of 0.17. [2, 1]

To complete: Add a matched ink-versus-shape design comparison when available; do not compare different writer populations as if only preprocessing changed. Verify the selected checkpoint's preprocessing in the final submitted configuration.

The width-selection tooling separates deterministic preparation from repeated learning. It skeletonises and redraws each crop once per candidate width, saving images together with writer

names, labels and split membership. Training then augments those prepared images each epoch. A fixed split and controlled initialisation make the widths comparable without changing the writer population at the same time. The preparation code records cases where normalisation returns no usable ink and the raw crop is retained; these fallbacks should be counted when interpreting a nominally shape-based dataset. [1, recognise.py; train.py; trainlibrary.py]

Table 3 records the supplied eleven-class validation sweep. All four runs had a 50-epoch budget and used the framework writer model on a Google Colab T4 GPU. The reported epoch is the best saved epoch, not the stopping epoch. The 0.17 setting gave the highest held-out accuracy in this sweep and defines the selected eleven-writer configuration; this is a preprocessing choice evaluated on validation data, not the integrated robot's writer-identification result. [2, training records]

Width fraction ρ	Best saved epoch	Held-out writer accuracy
0.06	38	98.06%
0.09	43	97.42%
0.11	39	98.06%
0.17	43	99.35%

Table 3. Stroke-width selection using the same eleven-writer population and held-out partition. [2, 1]

The selected checkpoint stores its width as 0.17. Both preparation and inference for that configuration must use this value; the module default of 0.14 is not its training setting. Figure 14 illustrates the selected transformation on existing crops. Reusing the held-out partition to choose width makes it model-selection data; an independent test set remains necessary for an unbiased final performance claim. [2, training records]

3.5.5 Training procedure and model selection

The implemented training routes update the convolutional backbone and task-specific head together. Convolution weights use variance-scaled initialisation; normalisation scales begin at one and offsets at zero. Training-time augmentation changes line geometry while preserving its label. The implemented transforms include shear, small rotations, perspective changes and elastic distortion. These operations are applied only to the training view, while validation retains a consistent preprocessing path. [1, recognise.py; train.py; trainlibrary.py]

The text trainer supports public lines, page-derived lines and a joint source that mixes public and personal examples. In the joint configuration, a personal-sample fraction controls their representation rather than letting a much larger public dataset determine the entire batch distribution. The default fraction is 0.20. This is a sampling choice, not a claim that personal and public samples are interchangeable. Its value and the training source must accompany any comparison between checkpoints. [1, recognise.py; train.py; trainlibrary.py]

Table 4 summarises the numerical training recipe exposed by the current implementation. The number of epochs is a run setting and should be reported from the actual training record, rather than taking a command-line default as the completed run. [1, recognise.py; train.py]

Setting	Reader	Writer classifier
Objective	CTC loss	Cross-entropy
Optimiser	RMSprop	AdamW
Initial learning rate	10^{-3} default	10^{-3} default
Weight decay	10^{-5}	10^{-4}
Learning-rate schedule	Plateau reduction, factor 0.5, patience 5, minimum 10^{-5}	Cosine annealing over the configured run
Gradient-norm clipping	Not applied by the current NumPy text loop	Maximum norm 5
Best-checkpoint criterion	Lowest validation loss	Highest validation accuracy

Table 4. Training settings implemented in the first-principles trainer. Final run settings require their own training records. [2, 1]

Cross-entropy penalises the writer model's predicted probability of the true class. For label y and probabilities p_k, its per-sample loss is $-\log p_y$, with logit derivative $p_k - \mathbf{1}[k = y]$ [18, p. 6, Eqs. (13)–(14)]. The CTC backward pass similarly relates each predicted frame probability to its target-state occupancy, calculated by forward and backward recurrences through the alignment lattice [19, PDF pp. 4–6]. These gradients are propagated through the task head and the convolutional stages; they are not supplied to the numerical trainer by automatic differentiation. [1, recognise.py; train.py; trainlibrary.py]

Validation runs with dropout disabled and stored normalisation statistics. A low training loss alone does not select the model. Separate checkpoint and best-model files preserve progress while retaining the selected validation model. Reproducible batching, declared splits and saved writer mappings support later evaluation, but comparable recipes do not imply identical weights or learning curves across implementations. [1, recognise.py; train.py; trainlibrary.py]

Algorithm 7 identifies the operations actually required for a first-principles training batch. Forward caches store quantities needed by the derivatives, rather than rerunning stochastic augmentation or dropout during the backward pass. The complete numerical batch/update procedure appears in Algorithm 7 in Appendix A.5; the calculations remain explicit in the source [1, train.py].

3.5.6 First principles and framework implementations

The first-principles route in `train.py` evaluates the forward pass, backward pass, loss and optimiser updates using explicit NumPy operations. NumPy supplies array arithmetic and matrix operations; it does not supply a convolutional network, automatic differentiation or a CTC training function in this route. The learned parameters are stored as named arrays. The inference module reads these arrays and evaluates the forward operations without requiring a machine-learning framework. [1, recognise.py; train.py; trainlibrary.py]

The framework route in `trainlibrary.py` expresses the corresponding networks using PyTorch layers and automatic differentiation. Both routes use the project's data preparation, parameter naming and named-array export layout. This makes the framework implementation useful as a numerical reference while allowing either training route to supply weights to the NumPy inference code. [1, recognise.py; train.py; trainlibrary.py]

The selected reader and eleven-writer weights were trained using PyTorch and exported to named arrays. The first-principles contribution comprises the implemented numerical training operations and the operational NumPy forward path. Fixed-parameter comparisons verify the numerical calculations; the provenance of the selected weights remains the framework training route [2, training records][1, recognise.py; train.py; trainlibrary.py].

Matching the architecture is not sufficient to establish a matched training experiment. The current numerical CTC function averages unnormalised sequence losses across the batch, whereas the framework trainer constructs CTC with its default mean reduction, which normalises each sequence by target length before averaging [22, CTCLoss, "Parameters"]. The framework loop also zeroes impossible-alignment losses. These conventions must be matched for a controlled training comparison because target-length weighting changes the influence of short and long lines. Named-array export remains compatible with both routes. [1, recognise.py; train.py; trainlibrary.py]

The reader checkpoint is `paper_cnn_bilstm_ctc_joint_best.npz`; the selected eleven-writer checkpoint is `author_shape_11_pw170_weights.npz`. The reader's training computer, completed epoch count and original learning-rate/batch settings are not preserved in the supplied record. The implementation defaults in Table 4 must therefore not be presented as recovered settings for that historical run. [1, recognise.py; train.py; trainlibrary.py]

The operation-by-operation comparison is retained in Table 7, Appendix A.7. [1, recognise.py; train.py]

Equation 17 and the shape progression in Table 2 define the implemented forward calculation. Its numerical correspondence and the backward operations are checked separately in Section 3.5.7. This separation distinguishes a correct description of an architecture from evidence that its calculations were implemented correctly. [1, recognise.py; train.py; trainlibrary.py]

The patch-matrix weight/input derivatives, their worked substitution and the numerical training-memory calculation are given in Appendix A.6 [1, train.py, Conv2dBackward].

3.5.7 Network verification and design limitations

Verification separates numerical correspondence from learning performance. For the first, the implementations receive the same parameters and inputs, with matched precision, normalisation state and stochastic masks. Differences in logits, loss and gradients then test the implementation. A second experiment trains each route on the same split with recorded initialisation and settings, and compares learning behaviour and held-out predictions. These experiments answer different questions: separately trained weights can differ even when the underlying operations agree. [1, recognise.py; train.py; trainlibrary.py]

The existing comparison archive contains a controlled text experiment using 500 training lines, 40 validation lines and 40 test lines over 15 epochs. It is useful supporting evidence for the implementation, but it is not the final eleven-writer system evaluation. A separate archived double-precision comparison uses two random 64 by 640 inputs and identical initial weights, checking the reader and a thirteen-output writer head. Table 5 summarises its recorded values. The writer dropout is disabled for this comparison so that unrelated random masks do not dominate the difference. [1, recognise.py; train.py]

Recorded comparison metric	Reader	Writer classifier
Maximum logit difference / maximum magnitude	2.12×10^{-14}	3.04×10^{-15}
Absolute loss difference	1.14×10^{-13}	1.78×10^{-15}
Worst ranked gradient relative L_2 difference	2.80×10^{-13}	6.01×10^{-14}

Table 5. Archived float64 implementation comparison from the CleanPipeline2 verification script. This is a controlled numerical check, not a final-model recognition result. [2, 1]

The logit metric in Table 5 divides the largest absolute difference by the larger maximum magnitude, with a 10^{-8} denominator floor. Gradient differences are $\|g_N - g_T\|_2 / \|g_T\|_2$, where g_N and g_T are numerical-route and framework gradients. Tensors whose maximum gradient magnitude is below 10^{-6} are retained in the archive but excluded from the worst-ranked summary: dividing roundoff by a nearly zero gradient gives an unhelpful relative error. These results support close agreement for that tested configuration; they do not establish equivalence for every input, code revision or training trajectory. [1, recognise.py; train.py]

To complete: Confirm applicability of the archived comparison to the submitted implementation and add the matched final training comparison. Obtain any missing final reader/writer comparisons from the existing project work.

A separate, newly run small-lattice check enumerated all $4^5 = 1024$ frame paths for a repeated-label target and compared their summed probability with the numerical CTC loss. The absolute loss difference was 4.44×10^{-16}; central differences over all 20 logits gave a maximum absolute derivative difference of 3.63×10^{-11}. This verifies that finite example, including the required blank between repeated labels. Its construction and limits appear in Appendix B; it is not a learned-model accuracy measurement. [2, 1, project records; implementation]

The finite-difference reference, numerical substitution and its treatment of non-smooth operations are recorded in Equation 40 in Appendix A.6.

The main design limitations concern what the input and training population can support. Fixed-width fitting limits detail in long lines; a closed writer head always selects among its trained classes; and line-level personal holdout does not establish independence from page or capture conditions. Stroke normalisation improves consistency with the pen plotter but can destroy detail when thresholding or thinning fails. These limitations should be assessed through appropriate examples and held-out measurements, rather than by treating a high softmax probability as proof of generalisation. Formal recognition and writer-identification qualification remains in Results. [1, recognise.py; train.py; trainlibrary.py]

3.6 Style extraction and handwriting reproduction

Recognising a line does not provide the motion needed to write it. The reproduction stage must recover usable pen paths from a writer's examples, store the properties that distinguish that writer, and assemble the paths into text that was not necessarily present in the examples. The chosen representation is a library of normalised glyph trajectories accompanied by line-level style measurements. It provides explicit control over scale and movement, while making the limits of character-based reproduction visible. [1, style.py; synthesise.py; reproduce.py]

3.6.1 Choice of reproduction representation

The available writer samples are a few labelled pages photographed after writing. A glyph library can reuse these observations without fitting a separate generative network for each writer. Haines et al. provide a relevant precedent: their standard experiments used approximately four A4 pages per writer [23, PDF p. 4, Sec. 4]. This is evidence that a small-page collection can support a glyph-based approach, not a guarantee that four pages cover every character or writing context. The present library likewise depends on the characters and variants actually observed.

The offline data also influenced the choice. Graves's recurrent synthesis model learns from ordered pen positions and end-of-stroke markers [18, p. 18]; these are not recorded in this project's photographs. A skeleton supplies geometric paths, but cannot recover their true order, speed or pressure. Using it as measured online training data would introduce an unsupported assumption. [2, 1, project records; implementation]

Learned image generators remain a valid alternative, including methods that need very few references from a new writer. For example, One-DM uses one style reference at generation, but its IAM experiment trains with 339 writers and its implementation uses four RTX 3090 GPUs [24, p. 9]. A small reference set must therefore be distinguished from the corpus and computation needed to train the generator. Developing and training an additional generator from first principles on the ODROID was outside the chosen computational scope; no training-time comparison was measured. The glyph approach instead retains explicit trajectories that can be scaled, joined and converted directly to machine coordinates. Its trade-off is weaker modelling of connected, context-dependent letterforms, examined at the end of this subsection. [1, style.py; synthesise.py; reproduce.py]

3.6.2 Character alignment and glyph extraction

The extraction input is a line image and its known transcript. Character boundaries are not supplied as labels. Instead, the trained reader gives a probability distribution for every output frame, and forced alignment finds the highest-scoring CTC path consistent with the supplied text. This is a localisation problem: the transcript constrains which labels must occur, while the frame scores indicate where they most plausibly occur. [1, style.py; synthesise.py; reproduce.py]

The alignment uses the same permitted transitions as the CTC training lattice, but maximises a path score instead of summing all path probabilities. Equation 23 gives the recurrence by replacing path summation in the CTC lattice with maximisation [19, PDF pp. 4–5].

$$V(t,s) = \log p_t(\ell'_s) + \max_{r \in \mathcal{P}(s)} V(t-1,r). \tag{23}$$

Calculated example: If $p_t(\ell'_s) = 0.7$ and the admissible predecessor log scores are -2 and -3, $V = \log(0.7) + \max(-2,-3) = -2.3567$. Extraction selects one predecessor; the CTC likelihood instead sums both. [1, recognise.py, CtcForcedAlign]

Here ℓ' is the blank-interleaved transcript and $\mathcal{P}(s)$ is the permitted predecessor set: stay, advance one state, or skip an admissible blank. Backtracking yields the frames assigned to each character. Equation 23 makes clear why correct text does not guarantee correct boundaries: several spatial allocations can remain plausible under the same transcript. [1, style.py; synthesise.py; reproduce.py]

An output frame is converted through the network canvas to the original crop using the inverse of its aspect-preserving scale. If a frame edge is t out of T and the crop was scaled by a, its initial horizontal location is approximately $(640t/T)/a$, clamped to the source width. This undoes the resize rather than assuming that every crop occupied the complete 640-pixel width. [1, style.py; synthesise.py; reproduce.py]

The baseline and x-height are operational estimates from the line's horizontal ink profile, not annotated typographic measurements. With $p(y) = \sum_x M(y,x)$, the implementation takes the first and last rows satisfying $p(y) \geq 0.45 \max p$ as the body-band top and baseline; their difference gives h_x. Bands below three pixels are rejected. This common scale makes glyph dimensions comparable across source resolutions, although unusual capitals, short lines or a large descender can bias the estimated body band. [1, style.py; synthesise.py; reproduce.py]

The worked example in Figure 15 uses an archived original Owen line crop, visually matched to the first line of its page transcript, with the joint reader checkpoint. The reader recovered the full line, and the extraction routine then generated the displayed boundaries and normalised paths. Correct text does not certify a correct individual cut: small punctuation and neighbouring strokes still require inspection. The archived crop is identified separately from the new whole-page segmentation run in the evidence manifest. [2, 1, project records; implementation]

The initial boundaries are refined by looking for thin, low ink near the expected cut. Equation 24 describes the principal column cost in `ExtractLineGlyphs`.

$$C_x = \frac{\overline{p}_x}{q_{0.60}(\{\overline{p}_j : \overline{p}_j > 0\})} + 1.15 \operatorname{clip}\left(\frac{b - y_{\text{top}}(x)}{h_x}, 0, 3\right). \tag{24}$$

Calculated example: For illustrative smoothed ink count 2, reference percentile 10 and $(b - y_{\text{top}})/h_x = 0.2$, $C_x = 2/10 + 1.15(0.2) = 0.43$. At the same ink count but body height 1.0, the cost rises to 1.35. [1, style.py, ExtractLineGlyphs]

Here $\overline{p}_x$ is smoothed column ink count, the denominator is its 60th percentile over positive counts, b is baseline position, and $y_{\text{top}}(x)$ is the highest ink row in that column. A denominator floor prevents division by zero. A true blank column is assigned zero cost. The search radius is approximately half an x-height, with a smaller penalty for distance from the initial boundary. The two terms in Equation 24 favour a narrow connector near the baseline over cutting through the body of a letter. [1, style.py; synthesise.py; reproduce.py]

One shared boundary array is enforced to remain increasing, with a minimum cell width of $\max(1, 0.08h_x)$ pixels. Independently snapping neighbouring boundaries could place both at the same gap and create an empty glyph. Figure 15 shows the refined cuts and resulting character shapes. [1, style.py; synthesise.py]

3.6.3 Skeleton tracing and pen path construction

Ink is an area, whereas the gantry needs a path. Zhang–Suen thinning reduces the line mask to a one-pixel centreline [21, p. 237]. The complete line is thinned before character cells are separated. Thinning each cut cell independently would introduce an artificial image edge at every character boundary and could change the recovered centreline there.

The skeleton is treated as a graph: adjacent ink pixels form edges, and endpoints and junctions determine where paths begin or split. Redundant diagonal edges are suppressed when an

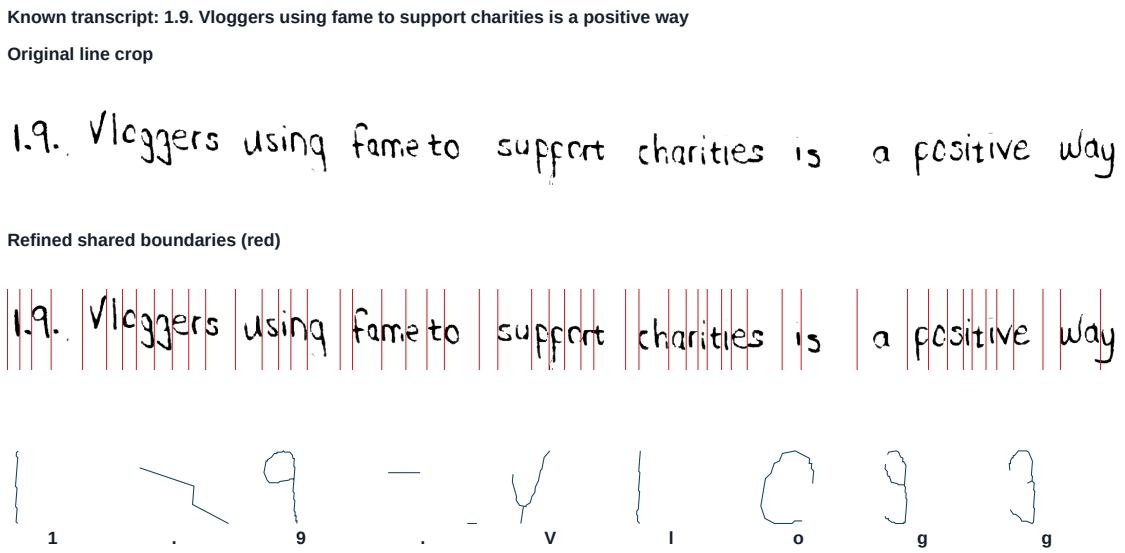


Figure 15. Transcript-constrained character localisation and refinement of glyph boundaries. [2, 1]

orthogonal route already connects the same local pixels. The default tracer first splits paths at junctions, prunes short dangling spurs, then greedily rejoins nearby ends with a continuation angle below 75°. Thus the default is not simply one pen lift per junction. A separate through-junction tracer exists but is not enabled by default. Neither procedure establishes natural pen order: the image records final ink, and several stroke sequences can create the same graph. [1, style.py; synthesise.py; reproduce.py]

Traced paths are divided at the character boundaries and simplified while limiting geometric deviation, following the polyline-simplification principle of Douglas and Peucker [25, page to verify] [*verify source/page:* `douglas1973`]. Short spurs and cut ligature tails require care because a genuine small stroke can resemble a skeleton artefact. Isolated dots may thin to a single point with no traceable edge; these are recovered as short paths rather than silently discarded. [1, style.py; synthesise.py; reproduce.py]

A detached `t` or `f` crossbar can extend over the next character cell. The extractor checks its width, flatness and height and, when the configured crossbar rule applies, assigns the complete bar to the corresponding letter instead of splitting it at a vertical cut. Such an overlap is not recorded as an in-word pen connection. Without this distinction, a crossbar would both contaminate the next glyph and inflate the profile's connectedness. [2, 1, project records; implementation]

The coordinate frame separates source-image size, baseline and slant from the geometry to be stored. Equation 25 first expresses each path point relative to the cut boundary x_L and baseline b, using the estimated x-height h_x and slant ϕ.

$$v = \frac{b-y}{h_x}, \qquad u_0 = \frac{x-x_L}{h_x} - v\tan\phi, \qquad u = u_0 - \delta. \tag{25}$$

Calculated example: With chosen $b = 40$, $y = 28$, $h_x = 24$, $x - x_L = 12$, $\tan\phi = 0.2$ and post-trim $\delta = 0.1$, $v = 0.5$, $u_0 = 0.5 - 0.5(0.2) = 0.4$ and stored $u = 0.3$. The separate lead is $l = 0.1$. [1, style.py, ExtractLineGlyphs]

Image rows increasing downwards become glyph height increasing upwards. After trimming eligible ligature tails, the code sets $\delta = \min(u_0 + v\tan\phi)$ over the remaining points, subtracts it from the stored horizontal coordinates and records lead $l = \text{clip}(\delta, 0, 1.5)$. Thus the stored path and its lead are separate quantities: they preserve the relation between the character cell and the ink rather than forcing every cell to start at its first stroke. Slant is removed for storage and reapplied during placement. The glyph can then be drawn at a chosen physical x-height instead of a fixed number of source pixels. [1, style.py; synthesise.py; reproduce.py]

Entry and exit positions are recorded along with whether the skeleton crosses a character boundary. This distinguishes a measured original connection from a merely small visual gap. The extracted paths still represent a geometric reconstruction, rather than the writer's true pen trajectory: retracing, pressure and movement direction cannot generally be recovered uniquely from a static image. [2, 1, project records; implementation]

3.6.4 Glyph selection and library quality

The library must retain variation without giving every cut equal credibility. A recogniser can assign the correct transcript while placing one boundary inside a letter, so alignment is followed by geometric and consistency checks. Low-confidence lines are filtered relative to the writer's own alignment distribution. Per-glyph checks reject implausible width, height, stroke count, pen length and disagreement between ink width and the allocated character advance. [1, style.py; synthesise.py; reproduce.py]

The width gate allows up to 2.1 x-heights for ordinary characters and 2.9 for wide characters such as `m` and `w`. Further width and height checks use the character's own distribution, rather than applying one absolute size to every letter. The 40th-percentile width provides a reference less easily enlarged by a few cuts that contain part of a neighbour. Tightening is retained only when enough variants survive to form a usable pool. These gates encode plausible glyph geometry; their exact values are engineering heuristics, not universal typographic laws. [1, style.py; synthesise.py; reproduce.py]

A cross-writer shape prior supplies a second check. Each variant is represented by a smoothed, normalised 12×12 occupancy descriptor. Pooling descriptors of the same character across writers gives a robust reference shape. Its role is to reject malformed cuts from a writer's own library, not to copy another writer's letter into it. Internal consensus then favours variants close to the writer's typical form. The ordinary variant pool is limited to twelve per character; an accepted denoised prototype can be prepended as an additional candidate. [1, style.py; synthesise.py; reproduce.py]

For unit-length descriptor v_g and the normalised median reference r_c for character c, the prior distance is $d(g,c) = 1 - v_g^T r_c$. Internal consensus instead uses pairwise L_1 descriptor distances and chooses the medoid with the lowest total distance. These are two distinct comparisons: resemblance to a pooled character reference and resemblance to this writer's own forms. Algorithm 4 gives their place in the complete profile-building procedure. For a chosen descriptor dot product of 0.9, prior distance is $1 - 0.9 = 0.1$. Three variants with pairwise L_1 distances 0.2, 0.6 and 0.5 have summed distances 0.8, 0.7 and 1.1, so the second is the medoid; it remains an observed glyph, rather than an averaged path. [1, style.py; synthesise.py]

Algorithm 4 Labelled handwriting to a persistent writer profile [2, 1]

Require: Labelled original lines $\mathcal{L}$, trained reader and writer identifier.

$\mathcal{G} \leftarrow \varnothing$; $\mathcal{S} \leftarrow \varnothing$
for each $(G, \ell) \in \mathcal{L}$ **do**
  $A \leftarrow \text{ForcedAlign}(G, \ell)$; estimate baseline, x-height and slant
  if alignment or body band is unusable **then continue**
  Refine shared increasing cuts; thin full line; trace skeleton paths
  Split paths at cuts; assign crossbars; recover untraced dots
  Normalise glyph coordinates; record advance, lead and boundary crossings
  Append glyphs to $\mathcal{G}$ and line measurements to $\mathcal{S}$
end for
Cache raw extraction; reject low-confidence lines relative to this writer
for each character's glyph pool $\mathcal{G}_c$ **do**
  Apply size, height-class, path-length and cell-consistency gates
  Refine size range; apply cross-writer prior and within-writer consensus
  Rank and smooth retained variants; construct medoid and eligible prototype
end for
return persistent profile containing $\mathcal{G}$ and robust summaries of $\mathcal{S}$

Two additional representatives support stable generation: a medoid that is an actual extracted example, and a denoised prototype formed from compatible variants. Prototype strokes are aligned and resampled before taking robust summaries; different stroke counts are not averaged indiscriminately. A prototype is admitted as an additional candidate only when the implemented shape-prior test improves sufficiently. The accepted and rejected variants in Figure 16 illustrate the distinction between retaining handwriting variation and retaining extraction errors. [1, style.py; synthesise.py]

Figure requested: One real writer's character variants, identifying retained forms, neighbour-contaminated or fragmented cuts, the medoid and any accepted denoised prototype. Include the rejection criterion.

`design-glyph-quality.pdf`

Figure 16. Example of glyph-library filtering and representative selection. [2, 1]

Synthesis preferentially samples well-formed variants and can favour a compatible entry height when joining. Where the requested character is unavailable, the code can use a case-related shape, a similar form, an anchor glyph or a simple fallback. These options preserve the ability to generate text, but they do not establish that the missing character was reproduced in the writer's own style. The trajectory records glyph-source information so that fallback use can be distinguished from genuine extracted forms. [1, style.py; synthesise.py; reproduce.py]

The final stored profiles supply the glyph geometry used during the demonstration. Figure 17 shows actual variants from Dylan's final library alongside its representative forms. These paths can be inspected independently of a photograph or a generated sentence, making it possible to distinguish the stored shape from later slant, jitter and connector operations. [1, style.py; synthesise.py]

This final `a` pool contains nine variants. The four displayed examples retain recognisable loop geometry while differing in closure and curvature. The medoid and median prototype remain

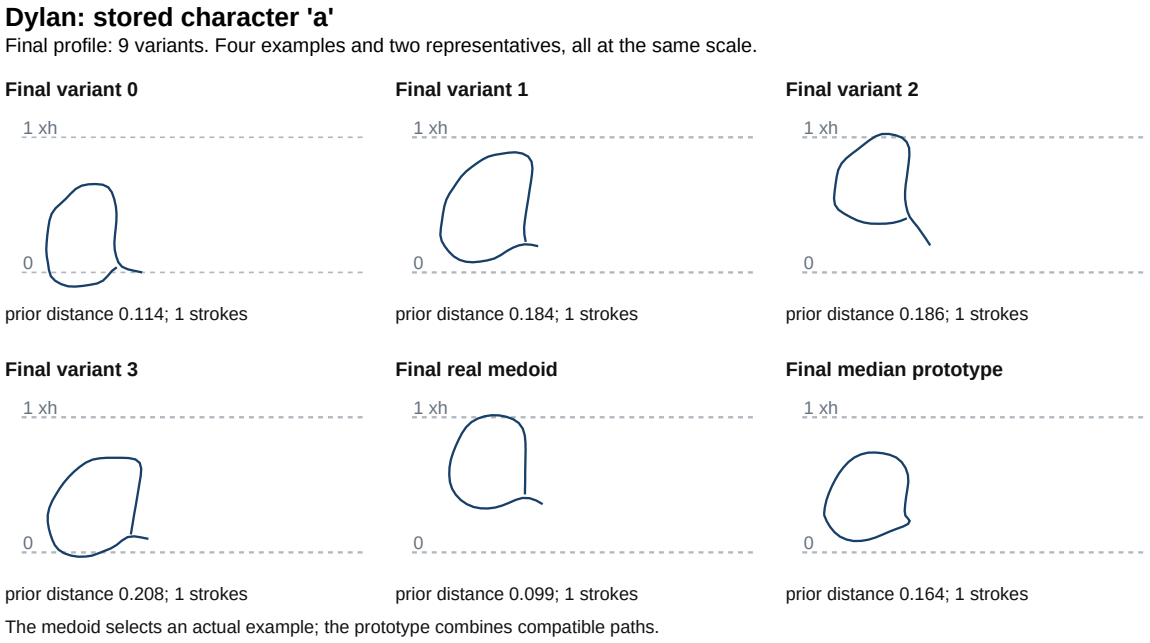


Figure 17. Actual glyph variants and representative forms in the final writer profile. [2, 1]

distinct: one selects a real variant, while the other combines compatible paths. A low prior distance expresses resemblance to the character reference, not proof of a better match to the writer. [1, style.py; synthesise.py]

3.6.5 Writer style measurement and storage

Letter shapes alone do not determine the appearance of a line. The profile also stores slant, x-height, ascender and descender extent, word spacing, per-character advance and connectedness. Measurements use medians where a few unusual cuts could otherwise dominate the result. Table 8 in Appendix A.7 gives the stored quantities, their units and their use in synthesis.

Slant is estimated by shearing the ink through candidate angles and measuring concentration of the resulting column profile. The stored angle is the one that most nearly aligns the dominant strokes. This treats slant as a property measured over a line, so it is less dependent on one unusual letter. It remains an estimate affected by letter content; it is not a direct observation of the writer's hand motion. [1, style.py; synthesise.py; reproduce.py]

Connectedness is measured using Equation 26.

$$c = \frac{N_{\text{cross}}}{\max(1, N_{\text{adj}})}. \tag{26}$$

Calculated example: If twelve of twenty eligible adjacent-letter pairs cross their source boundaries, $c = 12/20 = 0.60$. This illustrative fraction describes observed connections, not the proportion of future joins guaranteed to succeed. [1, style.py, BuildAuthorProfile]

Here N_{adj} counts eligible adjacent non-space glyph pairs and N_{cross} counts boundaries actually crossed by a traced centreline. Equation 26 measures connections in the source writing; counting pen lifts in a generated file would instead measure the extractor and synthesiser's decisions. The

difference is important when assessing how faithfully the library represents cursive writing. [1, style.py; synthesise.py; reproduce.py]

Profiles are stored as structured files containing the writer identifier, measurements and glyph paths. The permanent representation supports the proposal's multiple-writer requirement without re-extracting the source pages for each request. Cached raw extraction also permits a change to filtering without repeating all tracing. The extraction cache and final profile have different purposes: the former preserves intermediate work, while the latter is the version selected for reproduction. [1, style.py; synthesise.py; reproduce.py]

3.6.6 Text layout and connected handwriting synthesis

The synthesiser receives digital text, a profile, a physical x-height and a line-width limit. It selects a variant for each character, applies controlled shape variation and places the resulting paths into words and lines. Character advance and word spacing are taken from the profile rather than placing every letter at a fixed cell width. A word is moved to the next line when its predicted width exceeds the remaining space. [1, style.py; synthesise.py; reproduce.py]

Equation 27 expresses the main placement operation before machine-coordinate conversion.

$$x = p_x + h_x^{\mathrm{mm}}(l + u + sv), \qquad y = p_y + h_x^{\mathrm{mm}}(v + d). \tag{27}$$

Calculated example: Using the preceding glyph point, choose $h_x^{\mathrm{mm}} = 4$, $p_x = 10$, $p_y = 20$, $s = 0.2$ and $d = 0$. Then $x = 10 + 4(0.1 + 0.3 + 0.2 \times 0.5) = 12$ mm and $y = 20 + 4(0.5) = 22$ mm. [1, synthesise.py, SynthesizeText]

Here (u, v) is a stored glyph point, l is its lead-in offset, s is the applied slant shear, d is local baseline drift, and (p_x, p_y) is the placement origin. The physical x-height h_x^{mm} converts the normalised path to millimetres. Equation 27 keeps the source shape, line layout and physical scale as separate quantities. [1, style.py; synthesise.py; reproduce.py]

Small random translations, rotations and scale changes avoid using identical paths for every occurrence of a letter. A correlated drift term changes the baseline gradually instead of introducing unrelated vertical jumps. The implemented drift update uses a persistence factor of 0.85 and bounds displacement to ±0.25 x-heights. Such variation is controlled rather than unrestricted: too much jitter degrades letter identity and can cause neighbouring glyphs to collide. A collision guard maintains a nominal 0.05 x-height clearance where appropriate. [1, style.py; synthesise.py; reproduce.py]

Variant choice is weighted rather than uniform: the best-ranked forms receive greater probability, and a compatible entry height is preferred when joining. The legibility anchoring strength is also profile-dependent. In the automatic setting, Equation 28 maps the median stored prior distance $\widetilde{d}$ into a value λ.

$$\lambda = \mathrm{clip}\left(\frac{\widetilde{d} - 0.24}{0.32 - 0.24}, 0, 1\right). \tag{28}$$

Calculated example: An illustrative median prior distance $\widetilde{d} = 0.28$ gives $\lambda = (0.28 - 0.24)/0.08 = 0.5$; distances below 0.24 or above 0.32 saturate at zero or one. [1, synthesise.py, AutoLegibility]

A clean library therefore retains more sampled personal variants; a library with poor descriptor agreement uses stronger anchors. The anchor first prefers an acceptable representative from the writer's own profile, then uses a generic form when necessary. Equation 28 is a heuristic for

allocation of trust, not a learned probability that a glyph is correct. Source tags identify own, case-related, similar, core, fallback or blank output. In particular, “core” can represent either a personal representative or a generic anchor; that tag alone cannot quantify how much personal geometry was preserved. [1, style.py; synthesise.py; reproduce.py]

For connected writing, the profile’s connection fraction is used as a probability for eligible neighbouring letters. The code suppresses joining for profiles below a 0.45 connection threshold. A proposed join also requires a compatible glyph source and a gap no greater than 0.80 x-heights. This prevents a generic print fallback from being treated as a natural cursive attachment and prevents a long flat connector across an unusually large space. The body stroke is distinguished from a dot or crossbar so that the next letter does not connect to a detached mark. [1, style.py; synthesise.py; reproduce.py]

The default connector uses a shallow curved path between the exit and entry points, with sag limited relative to x-height and separation. A tangent-based alternative exists, but its flag is disabled in the current configuration. When a join is accepted, the connector and both bodies form one continuous pen-down path. Figure 18 relates endpoint selection to the resulting paths, including a pair for which a connector is unsuitable. [1, style.py; synthesise.py]

Figure requested: Enlarged real glyph pair with body, dot/crossbar, entry and exit labelled; show accepted connector and one rejected large-gap or incompatible-source case.

`design-glyph-joining.pdf`

Figure 18. Assembly of eligible glyph bodies into connected pen paths. [2, 1]

Line-level style calibration compensates for differences between measurements of individual stored glyphs and measurements of the assembled line. A probe is rendered without jitter, measured using the same style functions, and compared with the reference slant and vertical proportions. Limited shear and extent corrections are applied over three iterations. This improves internal consistency of the representation, but it remains calibration against the project’s own measurements; independent evaluation is needed to establish physical stylistic fidelity. [1, style.py; synthesise.py; reproduce.py]

3.6.7 Candidate selection for legibility and writer identity

One set of random variant choices can be more readable than another without being more characteristic of the writer. Candidate selection therefore evaluates both quantities. Each candidate trajectory is rendered at constant ink width, read by the text recogniser, and judged by the stroke-normalised writer classifier introduced in Section 3.5.4. Uniform rendering keeps candidate selection focused on reproducible geometry, while the shape classifier applies the selected stroke-normalisation convention. [1, style.py; synthesise.py; reproduce.py]

The score in Equation 29 combines text recovery and the requested writer’s probability.

$$A = \max\left(0, 1 - \frac{d_{\text{edit}}(\widehat{t}, t)}{|t|}\right), \qquad J = \begin{cases} 2AP/(A+P), & A+P>0, \\ 0, & \text{otherwise.} \end{cases} \tag{29}$$

Calculated example: For a 20-character target with two edit operations, $A = 1 - 2/20 = 0.90$. If the selected writer posterior is $P = 0.80$, $J = 2(0.90)(0.80)/1.70 = 0.8471$. This constructed score

explains selection and is not a measured stylistic-correlation result. [1, recognise.py, CharAcc; synthesise.py, SynthesizeJointBestOf]

Here t is intended text, $\widehat{t}$ is recognised text, d_{edit} is the minimum number of insertions, deletions and substitutions, using the reader error convention [19, PDF p. 2, Sec. 2.1], and P is the model probability of the selected writer. The harmonic mean J in Equation 29 penalises a weak component. For example, $A = 0.95$ and $P = 0.30$ give $J \approx 0.456$, whereas $A = P = 0.65$ give $J = 0.65$. An arithmetic mean would give the former candidate 0.625 and obscure much of its poor writer score. [1, style.py; synthesise.py; reproduce.py]

Algorithm 5 gives the bounded search and repair procedure. The server default is fourteen candidates, with the number configurable. Because an entire candidate is rerendered and rescored, a repair is judged in its assembled context rather than by an isolated character score. [1, style.py; synthesise.py]

Algorithm 5 Joint candidate search and bounded legibility repair [2, 1]

Require: Text, writer profile, reader, shape classifier, seed s and trial budget K.

$J^* \leftarrow -1$; $R \leftarrow \varnothing$; determine anchoring strength
for $k \leftarrow 0$ **to** $\max(1, K) - 1$ **do**
  $T \leftarrow \text{SynthesizeText}(\text{text}, \text{profile}, s + 977k)$
  $I \leftarrow \text{RenderUniformInk}(T)$; $(A, P, J) \leftarrow \text{Score}(I)$
  if $J > J^*$ **then** $(T^*, I^*, J^*) \leftarrow (T, I, J)$
  if $J^* \geq 0.99$ **then break**
end for
for at most three configured repair rounds **do**
  if $J^* \geq 0.995$ **then break**
  $W \leftarrow$ up to two weak aligned positions not already in R
  if $W = \varnothing$ **then break**
  $R' \leftarrow R \cup W$; regenerate with stronger anchors at R' and seed s
  Render and rescore the complete trial trajectory
  if $J > J^*$ **then** commit R' and the new best trajectory
end for
return T^* with selection and accepted-repair metadata

A repair is retained only when the complete trial improves the joint score. This bounds the search and protects the best candidate already found. A separate reader-only helper uses a different escalation policy; it is not part of this joint selection procedure. [1, style.py; synthesise.py]

More trials increase the opportunities to find a useful combination, but also add recogniser and writer-model calls. Search quality therefore has a computational cost for fixed text and profiles. The repaired or fallback glyph proportion also matters: legibility gained by replacing distinctive forms has a stylistic cost. Figure 19 relates the individual candidate scores to the selected trajectory. [1, style.py; synthesise.py]

In the worked Owen example, three pre-repair trials use the fixed text “Writing with style.”, seeds 2026, 3003 and 3980, and the selected eleven-writer shape model with an explicit width fraction of 0.17. Candidate 1 has $(A, P, J) = (0.947, 0.579, 0.718)$, while candidate 3 has the same character score but $(P, J) = (0.464, 0.623)$. The combined criterion therefore selects candidate 1. Its reader output begins with a lower-case “w”, which accounts for one edit rather than perfect text recovery. This is an inspectable scoring example before repair, not a claim that the full default search or the physical system attained those scores. [1, style.py; synthesise.py; reproduce.py]

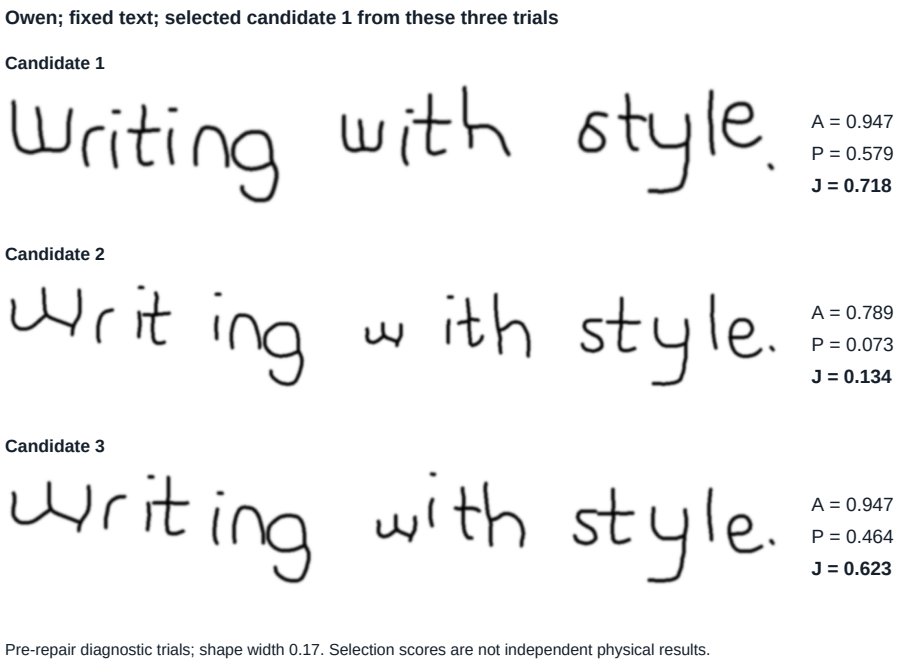


Figure 19. Example of joint candidate selection for a fixed writer, input text and search configuration. [2, 1]

The same models guide selection, so their scores cannot independently prove the output's quality. A classifier can prefer an image for model-specific reasons, and its posterior is not an 85% stylistic correlation measurement. The search is a design mechanism; final physical handwriting must be qualified separately. Increasing the search budget also does not recover a letterform that is absent or incorrectly extracted from the source library. [1, style.py; synthesise.py; reproduce.py]

3.6.8 Machine coordinate mapping and G-code generation

The selected trajectory is mapped into the calibrated workspace. Normalised glyph size, line spacing and machine margins are handled separately: fitting the height of a tall descender should not silently change the meaning of the ruling or move a line outside the available area. Robust extent measurements limit the effect of a single unusually tall glyph on the scale of the complete passage. [1, style.py; synthesise.py; reproduce.py]

The physical x-height is first chosen from the page ruling and a stored x-height-to-pitch ratio where available. Otherwise the code uses a fraction of 0.46, giving an illustrative $8.4(0.46) = 3.864$ mm x-height at the default 8.4 mm ruling. A probe of the actual requested text can reduce that value if robust line extent exceeds 1.25 rulings. The extent is measured using the 98th-percentile stroke tops and second-percentile bottoms, with exact extremes for fewer than eight strokes. It limits the influence of a few outlying strokes, but is not a guarantee that every extreme lies inside the nominal band. [1, style.py; synthesise.py; reproduce.py]

Two placement modes are supported. Ruled placement adjusts each line's measured centre to the configured ruling independently of the character scale. Free placement preserves the synthesised pitch after scaling, which can avoid forcing large ascenders and descenders from neighbouring

lines into contact. A line offset selects a later starting position and reduces the remaining page height. Algorithm 8 summarises the conversion before file generation. The full conversion procedure is retained as Algorithm 8 in Appendix A.9. [1, style.py; synthesise.py]

For the basic mapping, Equation 30 places a trajectory with bounds (x_0, y_0, x_1, y_1) within the page margins.

$$x_m = o_x + m_x + a(x - x_0), \qquad y_m = o_y - m_y - d_y - a(y_1 - y) + \Delta_y. \tag{30}$$

Calculated example: For a chosen origin $(0, 250)$ mm, margins $(15, 10)$ mm, scale $a = 1$, bounds $x_0 = 0$, $y_1 = 20$, and no line shifts, trajectory point $(5, 12)$ becomes $(x_m, y_m) = (20, 232)$ mm. These illustrative coordinates must still pass workspace checks. [1, reproduce.py, ToMachine]

Here (o_x, o_y) is the configured machine origin, m_x, m_y are margins, a is a fit scale, d_y selects the line offset and Δ_y is any line-spacing adjustment. Both stored trajectories and machine coordinates use upward-increasing y; Equation 30 positions later lines lower on the page rather than introducing another arbitrary image flip. Requested points are restricted to the usable travel, but clamping is a last boundary protection: it can distort a letter and must not be treated as successful layout. [1, style.py; synthesise.py; reproduce.py]

Strokes are ordered within text rows so that path shortening does not scramble reading order. Short path segments can be decimated, and nearby consecutive strokes may be joined without a lift. The current default distance is 0.6 mm for such joins and for closing a near-complete ring. These operations save travel and pen changes, but can also introduce unintended ink if set too aggressively; their scale relative to the chosen writing size matters. The code retains clipped-point and stroke information to make these effects inspectable. [1, style.py; synthesise.py; reproduce.py]

The generated file selects millimetres and absolute positioning. A travel move reaches the start of a stroke with the pen raised, the pen is lowered, drawing moves follow, and the pen is raised before the next travel. Figure 20 gives an illustrative 10 mm segment; it is not a claim of a measured hardware test. The interpreter assigns `M3` and `M5` to pen down and pen up respectively. [1, style.py; synthesise.py]

```
G21
G90
G0 F4000 X20.000 Y30.000
M3
G4 P0.350
G1 F800
G1 X30.000 Y30.000
M5
G4 P0.350
M2
```

Figure 20. Illustrative G-code sequence for positioning, lowering the pen, drawing one line and raising the pen. [2, 1]

Feed is modal, so the travel feed is restated after drawing rather than assuming that the first `G0 F` remains active. Pen dwell defaults to 350 ms in each direction and is configurable separately for raising and lowering. This dwell is additional to the switch-confirmed actuation performed by the

interpreter. A timing estimate that counted only geometric travel would omit those repeated state-change costs. Conversely, a trapezoidal acceleration parameter in a configuration object does not establish that the actual GPIO loop executes acceleration ramps. [1, style.py; synthesise.py; reproduce.py]

Equation 31 expresses a nominal job budget from the generated geometry.

$$t_{\text{nom}} = \frac{60D_d}{F_d} + \frac{60D_u}{F_u} + \frac{N_p\left(t_{\text{up}} + t_{\text{down}}\right)}{2}. \tag{31}$$

Calculated example: For chosen D_d = 100 mm, D_u = 50 mm, F_d = 800 mm/min, F_u = 4000 mm/min, twenty transitions and two 0.35 s dwells, $t_{\text{nom}} = 7.50 + 0.75 + 7.00 = 15.25$ s. It remains a nominal budget, not a measured job time. [1, reproduce.py, WriteGcode]

Here D_d, D_u are drawing and pen-up distances in millimetres, F_d, F_u their feeds in mm/min, N_p is the number of pen transitions and the dwell times are in seconds. The last term assumes alternating up/down events. Equation 31 separates settings that affect ink quality from settings that mainly affect non-writing time. It excludes synthesis, switch-confirmed motor actuation, GPIO overhead and the directional pulse-delay effect in Equation 5; the saved estimate is therefore not the measured R3 completion time. [1, style.py; synthesise.py; reproduce.py]

3.6.9 Reproduction verification and design limitations

The reproduction checks follow the representations established above: aligned source ink, stored glyph paths, assembled trajectories and emitted machine commands. Re-rendering the emitted G-code can reveal a coordinate inversion, scale change, missing stroke or additional joining segment before drawing. Comparing that render with the selected trajectory checks conversion; comparing the physical drawing with the render checks execution. These comparisons localise errors without substituting for end-to-end qualification. [1, style.py; synthesise.py; reproduce.py]

Appendix B shows the same sentence generated from four stored profiles alongside paths independently parsed from their emitted G-code. Nine controlled conversion fixtures additionally preserved path vertex counts with no clipping and a maximum coordinate discrepancy of 0.0004 mm, consistent with writing coordinates to three decimal places. Joins and ring closure were disabled only for those correspondence fixtures so that the vertices could be paired; the four profile illustrations used the normal configuration. Neither check measures the mechanism's positional accuracy. [1, style.py; synthesise.py; reproduce.py]

The representation's main advantage is that its geometry is explicit and inspectable. A defective glyph can be traced to a source crop, stored paths can be viewed independently of the machine, and physical size follows from a defined transformation. The main limitation is the loss of context when a connected word is divided into reusable characters. Real letterforms change with their neighbours, whereas a finite glyph pool and join rules approximate that dependence. Skeletons also omit pressure and true temporal order. These losses explain why a plausible letter library need not reproduce every connected style equally well. [1, style.py; synthesise.py; reproduce.py]

Possible extensions include context-dependent multi-character units, better alignment, trajectory acquisition and a verified alternative junction tracer. Each addresses a different loss: multi-character units preserve local letter interactions, trajectory acquisition restores temporal information, and improved tracing changes how an existing skeleton becomes strokes. Their

implementation would require corresponding changes to extraction, storage or the source data, rather than a larger candidate search alone. The retained design therefore prioritises extraction from the available offline pages and explicit control of the plotted geometry. Recovering natural pen dynamics would require new trajectory data, while replacing the library with a learned generator would add a separate training and validation problem. The alternative tracer remains an implementation option; its presence alone does not establish an improvement over the default. [1, style.py; synthesise.py; reproduce.py]

3.7 System integration and embedded operation

The server connects the numerical processing, models, profiles and motion interpreter. The browser supplies images or writing requests and displays their outputs; the engineering operations remain in the processing modules [1, web/server.py].

3.7.1 Classification and reproduction workflows

Captured and uploaded pages follow the same conditioning and recognition path. Writer predictions are combined across the retained line crops using Equation 32, rather than choosing one line to represent the page [1, web/server.py, NameWriter].

$$\overline{p}_k = \frac{\sum_i w_i p_{i,k}}{\sum_i w_i}, \qquad \widehat{k} = \arg\max_k \overline{p}_k. \tag{32}$$

Calculated example: two line weights 100 and 300 with writer probabilities 0.90 and 0.60 give $\overline{p}_k = [100(0.90) + 300(0.60)]/400 = 0.675$ [1, web/server.py, NameWriter].

Here w_i counts crop pixels below grey value 128; all-zero weights fall back to equal weighting. More ink therefore gives a line greater influence, including a possible dark artefact. Crop rejection precedes aggregation for this reason. The pooled probability and the fraction of individual lines agreeing with the selected writer are reported separately; neither establishes accuracy without a reference [1, web/server.py, NameWriter].

A reproduction request selects text, profile and drawing settings, then stores the trajectory, preview, G-code and metadata as one generated job. Characters absent from the profile use the fallbacks in Section 3.6.6. Drawing is a separate action requiring connection and calibration, allowing inspection before execution. Table 9 in Appendix A.7 records each representation's inputs, outputs and coordinate conventions [1, web/server.py; reproduce.py, WriteGcode].

3.7.2 Embedded execution, interfaces and job control

The ODROID N2+ runs NumPy inference and motion control from stored arrays, without the training framework. Canvas size, line count and candidate count still determine its processing load. NumPy supplies array arithmetic; Flask handles requests; image libraries handle file I/O; OpenCV serves the camera interface; and `gpiod` exposes GPIO. Framework network operations belong to the separate training route. The complete source code is supplied separately from this report for the AMS submission [1, recognise.py; trainlibrary.py; web/server.py; web/camera.py; reproduce.py].

During motion, the pulse loop checks an abort event and confirmed end stops. A separate request thread permits a stop while drawing is active. End-stop trips latch a fault; the recovery routines attempt switch retraction, pen raising and parking under their required preconditions. The stop request is a software path, not an independent motor-power disconnect. A pen-state timeout remains consequential because subsequent travel may mark the sheet [1, reproduce.py; web/server.py].

The resulting job states distinguish generation, refused execution, completed execution and interruption/failure. Calibration establishes the coordinate reference in software, but missed steps can invalidate its correspondence with physical position. Neither a saved preview nor emitted pulse counts prove that the nib followed the intended path [1, web/server.py; reproduce.py].

3.7.3 Integration verification and remaining constraints

Interface checks should confirm consistent upload/camera processing, checkpoint-to-profile writer mapping, bounded G-code and truthful interrupted-job status. Formal classification accuracy, drawing accuracy and timing against the proposal remain in Results [1, web/server.py; paths.py; reproduce.py].

The diagnostic retracing route provides a useful bypass: page conditioning and line separation are followed by thinning, tracing and ordinary machine conversion, omitting the reader, glyph library and synthesis. Comparing its G-code render with a drawing helps distinguish execution from synthesis errors. Comparing the drawing directly with the photograph still includes conditioning, skeletonisation, filtering and coordinate errors; it does not isolate mechanics alone [1, retrace.py, RetracePage].

Per-stage records separate acquisition, segmentation, inference, generation and execution time. Generation metadata is carried into the writing record so that a complete reproduction interval can be reconstructed. R3/R4 qualification must specify its start/end events and use the intended platform; Windows development timings cannot establish an ODROID requirement [1, timing.py; web/server.py].

4. References

[1] D. R. Braune, “Handwriting robot: CleanPipeline3 project source code,” Project repository snapshot, Software/CleanPipeline3, 2026, version inspected 1 November 2026; function locators identify unpaginated source.

[2] ——, “Handwriting robot: design records and subsystem verification evidence,” Unpublished project records accompanying the report source, 2026, hardware and training clarifications supplied 31 October–1 November 2026; PROJECT_DESIGN_RECORDS.md and Evidence manifests. Constructed examples are identified separately.

[3] O. Teikari, “Reproducing handwriting with position-controlled robots,” Master’s thesis, Tampere University of Technology, Tampere, Finland, 2016.

[4] Allegro MicroSystems, “A4988 DMOS microstepping driver with translator and overcurrent protection,” Datasheet, rev. 8, 2022, [Online]. Available: https://www.allegromicro.com/-/media/files/datasheets/a4988-datasheet.pdf.

[5] P. P. Acarnley, *Stepping Motors: A Guide to Theory and Practice*, 4th ed. London, U.K.: Institution of Electrical Engineers, 2002.

[6] J. E. Bresenham, “Algorithm for computer control of a digital plotter,” *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965. [Online]. Available: https://janmr.com/files/papers/bresenham65.pdf

[7] ROHM Semiconductor, “NPN transistor: Low-side switch fundamentals,” TechWeb, Transistors Basic, 2026, 27 February 2026; accessed 1 November 2026. Unpaginated; section headings used as locators. [Online]. Available: https://techweb.rohm.com/product/transistors-diodes/transistors/27354/

[8] ON Semiconductor, “BC337/BC337-40 NPN general-purpose amplifier transistor,” Datasheet, [Online]. Available: https://www.onsemi.com/download/data-sheet/pdf/bc337-fsc-d.pdf.

[9] F. C. Crow, “Summed-area tables for texture mapping,” in *Proc. 11th Annual Conf. Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1984, pp. 207–212.

[10] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[11] W. Postl, “Detection of linear oblique structures and skew scan in digitized documents,” in *Proc. 8th Int. Conf. Pattern Recognition (ICPR)*, 1986.

[12] S. Avidan and A. Shamir, “Seam carving for content-aware image resizing,” *ACM Transactions on Graphics*, vol. 26, no. 3, pp. 10:1–10:9, 2007. [Online]. Available: https://www.cs.jhu.edu/~misha/ReadingSeminar/Papers/Avidan07.pdf

[13] A. Rosenfeld and J. L. Pfaltz, “Sequential operations in digital picture processing,” *Journal of the ACM*, vol. 13, no. 4, pp. 471–494, 1966. [Online]. Available: https://www.cs.virginia.edu/~jlp/66.sequential.op.pdf

[14] F. Kizilirmak and B. Yanikoglu, “CNN-BiLSTM model for English handwriting recognition: comprehensive evaluation on the IAM dataset,” arXiv preprint arXiv:2307.00664, 2023. [Online]. Available: https://arxiv.org/pdf/2307.00664

[15] U.-V. Marti and H. Bunke, “The IAM-database: an English sentence database for offline handwriting recognition,” *Int. J. Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[16] K. Chellapilla, S. Puri, and P. Simard, “High performance convolutional neural networks for document processing,” in *Proc. 10th Int. Workshop on Frontiers in Handwriting Recognition (ICFHR)*, 2006.

[17] S. Ioffe and C. Szegedy, “Batch normalization: accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.pdf

[18] A. Graves, “Generating sequences with recurrent neural networks,” arXiv preprint arXiv:1308.0850, 2013. [Online]. Available: https://arxiv.org/pdf/1308.0850

[19] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 369–376. [Online]. Available: https://www.cs.toronto.edu/~graves/icml_2006.pdf

[20] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf

[21] T. Y. Zhang and C. Y. Suen, “A fast parallel algorithm for thinning digital patterns,” *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984. [Online]. Available: https://agcggs680.pbworks.com/f/Zhan-Suen_algorithm.pdf

[22] PyTorch contributors, “CTCLoss: reduction and alignment-loss conventions,” PyTorch documentation, accessed 1 November 2026. [Online]. Available: https://docs.pytorch.org/docs/2.15/generated/torch.nn.CTCLoss.html

[23] T. S. F. Haines, O. Mac Aodha, and G. J. Brostow, “My text in your handwriting,” *ACM Transactions on Graphics*, vol. 35, no. 3, pp. 26:1–26:18, 2016. [Online]. Available: https://reality.cs.ucl.ac.uk/projects/haines/haines16handwriting.pdf

[24] G. Dai, Y. Zhang, Q. Ke, Q. Guo, and S. Huang, “One-DM: one-shot diffusion mimicker for handwritten text generation,” in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science. Springer, 2024. [Online]. Available: https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/07598.pdf

[25] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,” *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[26] T. Tieleman and G. Hinton, “Lecture 6.5—RMSprop: divide the gradient by a running average of its recent magnitude,” COURSERA: Neural Networks for Machine Learning, 2012. [Online]. Available: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

[27] D. P. Kingma and J. Ba, “Adam: a method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/pdf/1412.6980

[28] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/pdf/1711.05101

A. Supporting implementation calculations

The main Design section explains the engineering decisions and representative operations. This appendix records the derivative conventions, image operations and parameter meanings needed to inspect the numerical implementation in more detail [1, `train.py`, `segment.py`, `style.py`]. Algorithms in the main text are explanatory pseudocode; the separately submitted source remains the executable implementation.

A.1 Numerical network derivatives

For a linear layer $Y = XW^T + b$ with incoming derivative G, the implemented derivatives are $\partial\mathcal{L}/\partial X = GW$, $\partial\mathcal{L}/\partial W = G^T X$ and the sum of G over samples for the bias. Convolution applies the corresponding matrix calculation to overlapping image patches as described in Equation 39. Contributions to the same input pixel are summed. [1, train.py; segment.py; style.py]

For each batch-normalisation channel, $N = BHW$ values contribute to the batch statistics. The training forward pass uses population variance, while the running variance update uses its $N/(N-1)$ corrected estimate. The running update momentum is 0.1. With $\widehat{x} = (x-\mu)/\sqrt{\sigma^2+\varepsilon}$ and $g = \partial\mathcal{L}/\partial y$, Equation 33 gives the input derivative, obtained by simplifying the batch-normalisation chain-rule expressions [17, PDF p. 4].

$$\frac{\partial\mathcal{L}}{\partial x_i} = \frac{\gamma}{N\sqrt{\sigma^2+\varepsilon}}\left[Ng_i - \sum_j g_j - \widehat{x}_i \sum_j g_j \widehat{x}_j\right]. \tag{33}$$

Calculated example: For three scalar inputs $(-1,0,1)$, $\gamma = 1$, $\varepsilon = 10^{-5}$ and incoming gradients $(1,0,0)$, the middle normalised input is zero. Its derivative is $-1/[3\sqrt{2/3+10^{-5}}] = -0.40825$, despite its own incoming gradient being zero. The shared statistics couple the samples. [1, train.py, BatchNorm2dBackward]

The parameter derivatives accompanying Equation 33 are $\sum_i g_i\widehat{x}_i$ for γ and $\sum_i g_i$ for β. Inference uses stored statistics and requires no batch-dependent update. [1, train.py; segment.py; style.py]

ReLU passes an incoming derivative only where its cached input was positive. Spatial max pooling scatters it to the selected maximum in each window. Height-max pooling does the same along height; width averaging distributes the derivative equally over the pooled columns. The implementation uses inverted dropout: it multiplies by the cached Bernoulli mask divided by $1-p$ during training and is the identity at inference. This moves the expected-output scaling from test time to training time relative to the original presentation [20, pp. 1931, 1934]. Reusing the same mask in the backward calculation is essential. [1, train.py; segment.py; style.py]

The LSTM backward pass differentiates the implemented recurrence in Equation 19; full back-propagation through time for this cell form is described by Graves [18, pp. 4–5]. It carries both the hidden derivative from the next step and a separate cell derivative. Writing the combined

hidden derivative as dh_t, Equation 34 shows the dependencies before gate-activation derivatives are applied.

$$\begin{aligned} dc_t &= dc_t^{\text{next}} + dh_t \odot o_t \odot [1 - \tanh^2(c_t)], \\ do_t &= dh_t \odot \tanh(c_t), & df_t &= dc_t \odot c_{t-1}, \\ di_t &= dc_t \odot g_t, & dg_t &= dc_t \odot i_t, \\ dc_{t-1} &= dc_t \odot f_t. \end{aligned} \tag{34}$$

Calculated example: Using the main-text scalar cell $c_t = 0.52$, $o_t = 0.7$, $f_t = 0.8$, with $dh_t = 1$ and $dc_t^{\text{next}} = 0$, gives $dc_t = 0.7[1 - \tanh^2(0.52)] = 0.5403$ and $dc_{t-1} = 0.4322$, before any other accumulated paths. [1, train.py, LstmBackward]

Sigmoid gates multiply these derivatives by $a(1-a)$; the candidate multiplies by $1 - g_t^2$. The concatenated gate derivatives accumulate outer products with x_t and h_{t-1} into the two weight matrices. Multiplication by W_h provides the next reverse-time hidden derivative in Equation 34. Each bidirectional layer reverses the appropriate sequence and sums the two input derivatives; stacked layers are differentiated in reverse layer order. [1, train.py; segment.py; style.py]

The numerical CTC forward score uses the recurrence in Equation 35, written in log space from the CTC path recursion [19, PDF p. 4, Sec. 4.1].

$$\alpha_{t,s} = \log p_t(\ell'_s) + \text{LSE}_{r \in \mathcal{P}(s)} \, \alpha_{t-1,r}. \tag{35}$$

Calculated example: For emission 0.7 and predecessor log scores $-2, -3$, $\alpha = \log(0.7) + \log(e^{-2} + e^{-3}) = -2.0434$. This exceeds the max-path score -2.3567 because both admissible paths contribute. [1, train.py, CtcLoss]

The predecessor set in Equation 35 contains the current state, the preceding state and an admissible two-state skip. A skip is excluded when its destination is blank or repeats the label two states earlier. With log scores a, b, the stable pairwise reduction is $m + \log(\exp(a-m) + \exp(b-m))$, where $m = \max(a, b)$; the all-unreachable case remains $-\infty$. The backward recurrence uses successor emissions at $t+1$, yielding the occupancy convention in Equation 21. [1, train.py; segment.py; style.py]

A.2 Optimisers and gradient scaling

The reader uses an uncentred RMSprop accumulator, following the moving-square-gradient principle [26, PDF p. 29]. The code selects its own averaging coefficient and adds the stated decay term. For parameter θ, raw gradient g, learning rate η and weight-decay coefficient λ_w, the implemented update is Equation 36.

$$g' = g + \lambda_w \theta, \quad v_t = 0.99 v_{t-1} + 0.01 (g')^2, \quad \theta_t = \theta_{t-1} - \eta \frac{g'}{\sqrt{v_t} + 10^{-8}}. \tag{36}$$

Calculated example: At the first update choose $\theta = 1$, $g = 0.2$, $\lambda_w = 10^{-5}$, $v_0 = 0$ and $\eta = 10^{-3}$. Then $g\prime = 0.20001$, $v_1 = 0.000400040001$ and $\theta_1 \simeq 0.990000$. The values illustrate the implemented update, not a recovered training step. [1, train.py, RMSprop]

Decay in Equation 36 enters the gradient before accumulation. The writer's AdamW instead uses separate parameter decay, as shown in Equation 37 [27, p. 2, Alg. 1][28, p. 3, Alg. 2].

$$\begin{aligned} m_t &= 0.9m_{t-1} + 0.1g, & v_t &= 0.999v_{t-1} + 0.001g^2, \\ \widehat{m}_t &= m_t/(1-0.9^t), & \widehat{v}_t &= v_t/(1-0.999^t), \\ \theta_t &= \theta_{t-1} - \eta\left(\frac{\widehat{m}_t}{\sqrt{\widehat{v}_t}+10^{-8}} + \lambda_w\theta_{t-1}\right). \end{aligned} \tag{37}$$

Calculated example: With zero initial moments, $g = 0.2$, $\theta_0 = 1$, $\eta = 10^{-3}$ and $\lambda_w = 10^{-4}$, $m_1 = 0.02$, $v_1 = 0.00004$, $\widehat{m}_1 = 0.2$, $\widehat{v}_1 = 0.04$ and $\theta_1 \simeq 0.9989999$. [1, train.py, AdamW]

Before the writer update, all gradient tensors are scaled together by $\min(1, 5/\|g\|_2)$, with a numerical denominator guard. Clipping individual tensor norms independently would be a different operation. The numerical reader loop does not apply that clipping. Equations 36 and 37 describe code operations, not a claim that their default learning rates were used in every completed run. [1, train.py; segment.py; style.py]

A.3 Morphology, thinning and path simplification

For binary mask M and neighbourhood K, dilation is the maximum of translated mask values and erosion is their minimum. Closing applies dilation followed by erosion with the same footprint. The implementation obtains rectangular neighbourhood counts through prefix sums; binary comparisons then implement these operations. Connectivity analysis uses horizontal runs and union operations with diagonal overlap admitted, rather than a library component-labelling call. [1, train.py; segment.py; style.py]

The thinning conditions follow Zhang and Suen [21, p. 237]. The method inspects the eight neighbours $P_2, \ldots, P_9$ clockwise from north. A removable foreground pixel has two to six foreground neighbours and exactly one background-to-foreground transition around that ring. The two alternating subpasses additionally require, respectively,

$$\begin{aligned} P_2P_4P_6 = 0,\ P_4P_6P_8 = 0 \quad &\text{(first subpass)}, \\ P_2P_4P_8 = 0,\ P_2P_6P_8 = 0 \quad &\text{(second subpass)}. \end{aligned} \tag{38}$$

Calculated example: For neighbours $(P_2, \ldots, P_9) = (1,1,1,0,0,0,0,0)$, the neighbour count is three, the circular transition count is one and all listed products are zero. A foreground centre therefore qualifies for deletion in the first subpass; the next subpass uses the updated image. [1, segment.py, Skeletonize]

Candidates in Equation 38 are deleted together in each subpass, and the pair of subpasses repeats until unchanged. This reduces thickness while limiting topology changes; it cannot restore ink already lost during thresholding. The same thinning primitive supports stroke normalisation, glyph tracing and the diagnostic retracing route. [1, train.py; segment.py; style.py]

The default tracer begins at graph nodes whose degree differs from two, follows unused pixel edges, and then recovers remaining closed loops. Short dangling spurs are pruned, and nearby chain ends are greedily rejoined when their continuation angle is below 75°. This remains a geometric estimate rather than a verified natural handwriting order. Polyline simplification compares interior points with the line through the endpoints, retaining a point and splitting recursively when the deviation exceeds tolerance. The code adapts the polyline reduction method [25, page to verify] [*verify source/page:* `douglas1973`]. Near-closed paths are first

split at a distant point so that a zero-length endpoint chord does not collapse the loop. The extraction tolerance is 0.6 source pixels before normalisation; final path smoothing and machine decimation occur in their own coordinate units. [1, train.py; segment.py; style.py]

A.4 Profile and trajectory quantities

Table 6 records the fields most likely to be confused during inspection. Full arrays and every tuning constant remain in the source rather than expanding the main text into a variable dictionary.

Quantity	Meaning and unit
`xHeightPx`	Reference source x-height in pixels; not a physical pen height
`strokes`	Ordered point lists in glyph x-height units; y increases upwards
`width`, `lead`, `advance`	Ink width, lead-in and cell/placement extent; distinct x-height quantities
`top`, `bot`	Vertical extrema relative to the baseline, in x-heights
`connL`, `connR`	Observed boundary crossings during source extraction
`priorD`	Descriptor distance from the pooled character reference; not writer confidence
`idealMedoid`, `idealGlyphs`	Actual representative variant and compatible median-stroke prototype
`letterAdvance`	Per-character median source advance; sampled variants retain their own placement geometry
`lineStepMm`	Synthesised baseline spacing before optional machine respacing
`jointScore`	Internal harmonic selection score; not independent stylistic correlation
`clipped`	Count of points clamped by the motion-file conversion; not an acceptable layout target

Table 6. Selected implementation fields and their interpretation. [2, 1]

Raw alignment confidence is a path score used for relative filtering, not a calibrated probability of a correct cut. The cross-writer prior uses a median occupancy descriptor, then unit normalisation; its minimum pooled sample count is twelve. Writer consensus retains approximately 75% of a sufficiently populated pool, with at least three variants. Compatible prototypes group equal stroke counts, resample strokes to 44 points, align them and take robust pointwise summaries. Admission to the variant pool requires prior distance below 0.32 and improvement of more than 0.02 over the median raw distance. These gates reduce malformed geometry but do not prove that a distinctive form is incorrect. [1, train.py; segment.py; style.py]

Machine conversion maintains image, glyph, trajectory and machine coordinate frames separately. The line-centre shifts in Equation 30 are zero when no respacing is requested. Workspace fitting and clipping are distinct: fitting scales the whole trajectory uniformly, while clipping changes individual points and can deform letters. Final file parsing provides a useful conversion check because it reconstructs movements from emitted commands instead of directly displaying the input trajectory again. [1, train.py; segment.py; style.py]

A.5 Detailed training procedures

The main chapter retains the model, loss, design choices and representative checks. The following pseudocode records the detailed recurrences and update sequence [1, train.py].

Algorithm 6 Numerical CTC loss and logit gradient [2, 1]

Require: Logits $z \in \mathbb{R}^{T \times B \times C}$ and encoded targets.

$\log p \leftarrow \mathrm{LogSoftmax}(z)$; $L \leftarrow 0$; $g \leftarrow 0$
for each batch item b **do**
  $\ell' \leftarrow$ target interleaved with blanks
  Initialise α, β to $-\infty$; admit first blank and first label
  for $t \leftarrow 1$ **to** $T-1$ **do**
    for each state s **do**
      $\alpha_{t,s} \leftarrow \log p_t(\ell'_s) + \mathrm{LSE}_{r \in \mathcal{P}(s)}\, \alpha_{t-1,r}$
    end for
  end for
  $\log Z \leftarrow$ LSE of the admissible final states
  Initialise final β to zero; recurse backward with next-frame emissions
  $q_{t,k} \leftarrow \sum_{s:\ell'_s=k} \exp(\alpha_{t,s} + \beta_{t,s} - \log Z)$
  $L \leftarrow L - \log Z$; $g_{t,b,k} \leftarrow (p_t(k) - q_{t,k})/B$
end for
return $L/B, g$

Algorithm 7 One numerical training batch and validation checkpoint decision [2, 1]

Require: Training batch, task, parameters θ and optimiser state.

$(X, y) \leftarrow$ task-specific preparation and training augmentation
$(F, \mathcal{K}) \leftarrow \mathrm{BackboneForward}(X, \theta)$; retain caches
if task is reader **then**
  $(z, \mathcal{K}_h) \leftarrow \mathrm{BiLSTMHead}(F)$; $(L, g) \leftarrow \mathrm{CTC}(z, y)$
else
  $(z, \mathcal{K}_h) \leftarrow \mathrm{WriterHead}(\mathrm{MeanWidth}(F))$
  $(L, g) \leftarrow \mathrm{CrossEntropy}(z, y)$
end if
$g_F \leftarrow \mathrm{HeadBackward}(g, \mathcal{K}_h)$
$\nabla\theta \leftarrow \mathrm{BackboneBackward}(g_F, \mathcal{K})$
if task is writer **then** clip total gradient norm to 5
$(\theta, \mathrm{state}) \leftarrow \mathrm{OptimiserStep}(\theta, \nabla\theta, \mathrm{state})$
if epoch is complete **then**
  Validate with fixed preparation, stored normalisation and dropout disabled
  Update schedule; save progress; replace best only if its criterion improves
end if
return $L, \theta, \mathrm{state}$

A.6 Expanded numerical calculation checks

For one image, let X_c contain flattened input patches, so the convolution becomes $Y = WX_c + b$. With output derivative G, Equation 39 supplies the two matrix derivatives used by the numerical route.

$$\frac{\partial \mathcal{L}}{\partial W} = GX_c^T, \qquad \frac{\partial \mathcal{L}}{\partial X_c} = W^T G. \tag{39}$$

Calculated example: For a reduced two-input patch $X_c = (1,2)^T$, kernel $W = (0.1, 0.2)$ and incoming derivative $G = 3$, the derivatives are $GX_c^T = (3,6)$ and $W^T G = (0.3, 0.6)^T$. Overlapping patches add their contributions to shared input pixels. [1, train.py, Conv2dBackward]

Weight and bias derivatives are summed over the batch and output positions. Overlapping patch derivatives in Equation 39 are added back into the padded input, not overwritten. Max-pool derivatives return to the recorded maxima, width-average derivatives are distributed uniformly, and the LSTM backward pass accumulates both hidden-state and cell-state contributions through time. Appendix A records the remaining derivative conventions and optimiser calculations without repeating the architecture here. [2, 1, project records; implementation]

The matrix formulation also has a memory cost. The first convolution's patch matrix contains $B(1)(3)(3)(64)(640)$ values. At batch size eight, its float32 storage alone is approximately 11.25 MiB, before later-layer patches, activations or gradients. This calculated example explains why numerical array operations do not automatically imply a small training footprint. Training memory and the embedded forward-only inference footprint must be assessed separately. [1, recognise.py; train.py; trainlibrary.py]

The finite-difference reference uses Equation 40, with one logit or parameter perturbed at a time.

$$g_i^{\text{num}} = \frac{\mathcal{L}(\theta + \varepsilon e_i) - \mathcal{L}(\theta - \varepsilon e_i)}{2\varepsilon}. \tag{40}$$

Calculated example: For the controlled scalar loss $\mathcal{L}(\theta) = \theta^2$, $\theta = 2$ and $\varepsilon = 10^{-4}$, the estimate is $[(2.0001)^2 - (1.9999)^2]/0.0002 = 4$, matching 2θ. This demonstrates the reference calculation, not a new whole-network gradient test. [2, `Evidence/bench.json`, CTC check]

Here θ is a parameter vector, e_i selects one component and ε is a small perturbation. Equation 40 requires care at ReLU or max-pool boundaries and with stochastic dropout. The input or mask must be controlled so that the two loss evaluations test the same local operation. [1, recognise.py; train.py; trainlibrary.py]

A.7 Implementation and interface reference tables

These tables preserve supporting implementation distinctions and representation contracts. The main chapter explains the corresponding choices and transitions [1, recognise.py; train.py; style.py; web/server.py].

Aspect	First principles route	Framework route
Layer calculations	Explicit forward and backward operations	Framework modules and automatic differentiation
Architecture	Project's reader and writer definitions	Corresponding layer pattern and parameter names
Data preparation	Project crop, label and split functions	Uses the same preparation functions
Weights for inference	Named array archive	Export to the same named array layout
Verification role	Implementation being checked	Reference for matched numerical comparisons

Table 7. Roles of the numerical and framework implementations. [2, 1]

Quantity	Source	Use in synthesis
Slant	Projection concentration under trial shears	Reapplied to stored upright glyphs
X-height	Height of the estimated line body band	Normalises stored paths; physical x-height sets drawing size
Ascender/descender extent	Relative glyph heights	Vertical proportions and line spacing
Letter advance	Character-cell and ink geometry	Placement of following characters
Word spacing	Measured gaps in real line ink	Separation of words
Connectedness	Observed skeleton crossings between adjacent characters	Probability of eligible in-word joins
Glyph variants	Filtered normalised trajectories	Writer-specific letterforms and variation

Table 8. Principal stored style quantities and their roles. [2, 1]

Stage	Required input	Output and governing convention
Segmentation	Photographed or saved page	Ordered raw crops and full-page boxes; image y points down
Recognition	Line crops and model arrays	Text plus writer probabilities interpreted by the checkpoint mapping
Profile construction	Original lines, known transcripts and reader	Writer-keyed glyph paths in x-height units, baseline origin and y up
Synthesis	Text, stored profile, scale and search settings	Millimetre trajectory, source tags and internal selection scores
File conversion	Trajectory and workspace configuration	Absolute millimetre G-code, pen events, placement and timing budget
Physical execution	G-code, connected GPIO and completed calibration	Step-counter updates and job outcome; physical accuracy requires observation

Table 9. Representation boundaries that connect the three main processing stages to the gantry. [2, 1]

A.8 Submission completion record

To complete: Confirm the completion entries against the final submitted deliverables, and identify the computer used for training and profile construction. The proposal's embedded-processing requirement must remain distinguishable from the actual training arrangement.

A.9 Motion-file conversion procedure

Algorithm 8 Trajectory placement and motion-file preparation [2, 1]

Require: Selected trajectory T and calibrated workspace configuration.

Measure bounds; determine available width and height after margins and offset
$a \leftarrow$ uniform fit scale; apply robust height cap when ruled fitting is enabled
Group paths by text row; determine ruled or free baseline shifts
$\mathcal{P} \leftarrow$ machine coordinates from Equation 30
Decimate short segments; optimise path order within rows
Clamp remaining out-of-bounds points; record their count
Emit absolute millimetre mode and initial pen state
for each path $p \in \mathcal{P}$ **do**
    Close eligible near-ring; join previous path if configured gap permits
    if a new pen-down path is needed **then**
        Raise pen; travel to start; lower pen; emit configured dwells
    end if
    Emit drawing moves with modal drawing feed; accumulate ink and travel budgets
end for
Raise pen; save file, placement and separate timing budgets
return file path and job metadata

B. Subsystem visual evidence and independent software checks

This appendix records newly run offline checks and illustrations made from existing project data. It supplements the Design explanations; the integrated qualification tests against the proposal remain in Results. The evidence scripts use the current processing modules, record source and input SHA-256 hashes, and write their outputs into this revision's `Evidence` directory. No training or physical motion was performed. Diagnostic settings are stated explicitly rather than inherited silently from server defaults. [2, subsystem manifests]

B.1 Actual capture variation and input resolution

The three camera-roll inputs in Figure 12 are `WIN_20261028_23_23_30_Pro.jpg`, `WIN_20261028_23_28_34_Pro.jpg` and `WIN_20261028_23_32_40_Pro.jpg`. Each stored image is 3264×2448 pixels. The page visibly contains three handwritten text rows. Automatic axis decisions were retained, including the failure in the third capture. The observed crop counts were 4, 3 and 28, respectively. Shadows are visually different, but neither lux levels nor physical angles were measured. These observations therefore explain a design limitation rather than qualifying operation across a specified lighting range. [2, subsystem manifests]

The native Owen01 page (2448×3264) produced 20 crops for 20 labelled lines. A separate exploratory bench first fitted each of three original pages inside 1600×1600 pixels. From each reduced page it produced an unchanged view, a synthetic left-to-right multiplier from 0.42 to 1.00, a synthetic +3° rotation using bicubic interpolation on an expanded white canvas, and a quarter turn. Each was passed through the same automatic axis and segmentation functions. Table 10 retains every case, including unsuccessful ones. [2, subsystem manifests]

Page	Label lines	Reduced	Gradient	+3°	90°
Owen	20	19	20	11	19
Dylan	24	24	24	25	24
Yeukita	26	26	26	26	26

Table 10. Exploratory crop counts after extra input reduction and digital perturbation. Equal counts do not establish correct ink ownership. [2, 1]

The reduced Owen case produced 19 crops and lost the usable first-line crop, whereas the native page produced 20. Its rotated reduced image produced only 11. The gradient image happened to restore the count to 20, which is not evidence that adding a shadow improves segmentation: thresholds, interpolation and component geometry changed together. These cases show why nominally scale-relative thresholds do not guarantee resolution invariance. A suitable follow-up would compare matched stroke ownership at controlled resolutions and improve capture guidance; the present bench does not supply pixel-level annotations or attribute every error to one operation. [2, subsystem manifests]

B.2 Independent component and seam references

Algorithm 9 states the two independent reference comparisons. The run-based eight-connected component labeller was compared with an independently written flood-fill reference on 100 binary masks of size 9×11, using NumPy seed 2026 and foreground probabilities spanning 0.1–0.8. The test compared the full same-component relation between foreground pixel pairs, not arbitrary numerical label values. All 100 partitions agreed. Small random masks exercise diagonal contact and label equivalences, but do not demonstrate large-image speed or prove every possible topology. [2, subsystem manifests]

For the seam solver, 20 random 4×6 cost arrays (seed 83) were checked against exhaustive enumeration of all row sequences with adjacent row changes of at most one. Both objectives included the implementation's $0.02|\Delta y|$ penalty. The maximum objective discrepancy was 0. The exhaustive reference tests optimal cost on those arrays; it does not establish that the chosen image-derived cost always follows the correct handwritten line boundary. [2, 1, project records; implementation]

Algorithm 9 Independent partition and seam verification [2, 1]

Require: Fixed random seeds, mask cases and small seam-cost arrays.

for each binary mask M **do**
  $A \leftarrow \mathrm{LabelComponents}(M)$; $B \leftarrow \mathrm{FloodFill8}(M)$
  assert $[A_p = A_q] = [B_p = B_q]$ for every foreground pair (p, q)
end for
for each small cost array C **do**
  $p^* \leftarrow \mathrm{LeastCostSeam}(C, 1)$
  $\mathcal{P} \leftarrow$ all paths with adjacent row differences at most one
  $E(p) \leftarrow \sum_x C(p_x, x) + 0.02 \sum_x |p_x - p_{x-1}|$
  Record $|E(p^*) - \min_{p \in \mathcal{P}} E(p)|$
end for
return partition agreements and seam-cost discrepancies

B.3 CTC probability and derivative check

Algorithm 10 gives the exhaustive probability and derivative procedure. The loss check used float64 logits of shape $5 \times 1 \times 4$, blank index 0 and target $[1, 1]$. All 1024 frame paths were enumerated. Consecutive repeats were collapsed before blanks were removed, and only paths yielding the target contributed to its probability. This independently exercises the repeated-label restriction that a single uninterrupted run of label 1 cannot encode two labels. The path sum gave negative log likelihood 3.875974036731708; the dynamic programme gave 3.875974036731709. The absolute difference was 4.44×10^{-16}. [2, subsystem manifests]

All 20 logits were perturbed individually with central-difference step 10^{-5}. The maximum absolute discrepancy from `train.py::CtcLoss` was 3.63×10^{-11}. Logits were drawn after the fixed-seed component-mask draws in the bench; the script preserves that order. These checks concern the implemented CTC loss and derivative on a finite feasible target. They do

not test impossible alignments, every transcript length, trained recognition accuracy or the full CNN–LSTM backward pass. [2, 1, project records; implementation]

Algorithm 10 Exhaustive CTC likelihood and finite-difference check [2, 1]

Require: Small logits z, target ℓ, blank index and step ε.

$(L, g) \leftarrow \mathrm{CtcLoss}(z, \ell)$; $p \leftarrow \mathrm{Softmax}(z)$; $Z \leftarrow 0$
for each frame path $\pi \in \{0, \ldots, C-1\}^T$ **do**
  if $\mathcal{B}(\pi) = \ell$ **then** $Z \leftarrow Z + \prod_t p_t(\pi_t)$
end for
Record $|L + \log Z|$
for each logit index i **do**
  $g_i^{\mathrm{ref}} \leftarrow [L(z + \varepsilon e_i) - L(z - \varepsilon e_i)]/(2\varepsilon)$
  Record $|g_i - g_i^{\mathrm{ref}}|$
end for
return loss discrepancy and maximum derivative discrepancy

B.4 Stored profiles, emitted files and physical page

Figure 21 uses the final Owen, Dylan, Yeukita and IAM 150 profiles with the same text, seed 2026, requested x-height 4 mm and jitter 0.15. The left column shows generated pen paths. The right column reconstructs pen-down paths from each emitted G-code file with the independent parser; each panel is fitted separately for legibility. The normal converter was used, including its path ordering, joins and near-ring closure. All four generated jobs had coordinates within the configured bed bounds. The figures illustrate profile-dependent geometry and file conversion, rather than measuring similarity to a human writer.

Algorithm 11 describes the serialization check. Nine controlled fixtures combined scale factors 1, 3 and 30 with line offsets 0, 8.4 and 25.2 mm. Ring closure and joining were disabled, minimum-segment filtering was set to zero, and the independently parsed paths were paired with `ToMachine`'s output. Each fixture retained path and vertex counts, with no clamped points. Maximum coordinate error was 0.0004 mm, within the 0.0005 mm per-coordinate rounding bound from three-decimal emission. This comparison checks serialization after placement; it does not independently prove that the placement equation itself is correct or that a motor reached a coordinate. [2, 1, project records; implementation]

Algorithm 11 Controlled G-code serialization correspondence [2, 1]

Require: Known multi-path trajectory and configurations with joins and closure disabled.

for each fixture configuration c **do**
  $P \leftarrow \mathrm{ToMachine}(T, c)$
  $f \leftarrow \mathrm{WriteGcode}(T, c)$; $Q \leftarrow \mathrm{ParseGcode}(f)$
  Compare path counts and corresponding vertex counts
  Record $\max_i \|P_i - Q_i\|_\infty$ and clamped-point count
end for
return count agreements, rounding discrepancies and clipping records

Figure 22 is separately identified as a photograph of actual robot output, from the existing Owen reproduction sheet. The line crops are newly obtained from that stored photograph. This closes

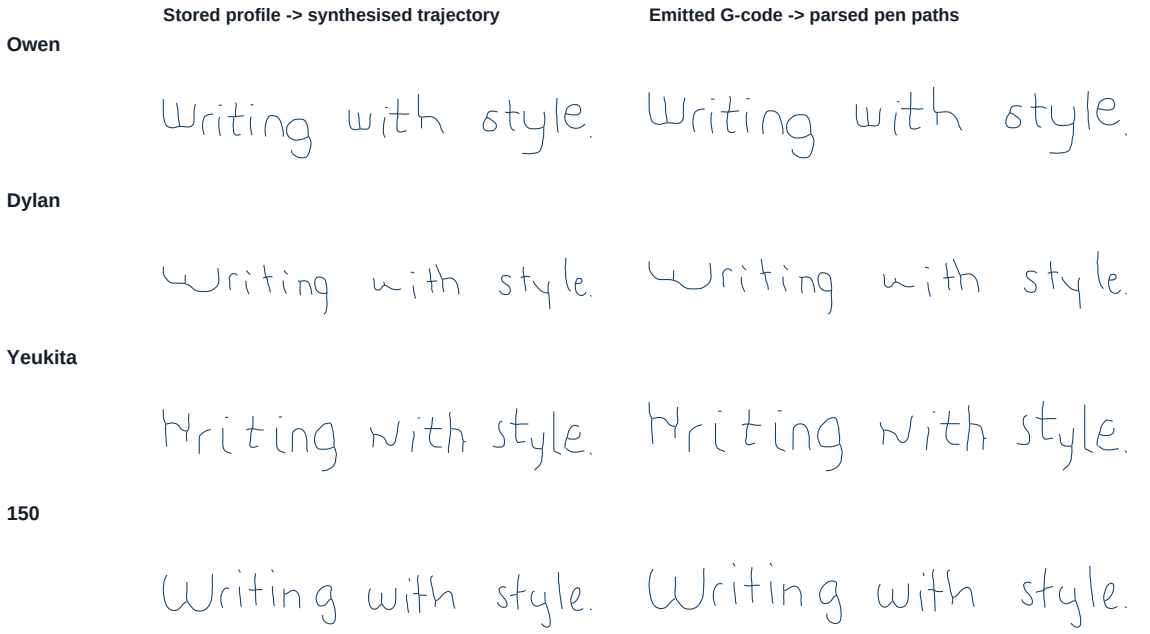


Figure 21. Same text generated from four final profiles and reconstructed from emitted G-code. These are software path drawings. [2, 1]

the visual explanation from a physical page to the segmentation interface, without presenting the earlier drawing as a newly executed hardware experiment. No exact same-job correspondence with Figure 21 is claimed. [2, subsystem manifests]

B.5 Candidate-scoring configuration and scope

The Owen trials use the joint reader and eleven-writer `pw170` checkpoint, with explicit 0.17 normalisation and uniform-ink rendering. They score character accuracy and requested-writer posterior before repair, independently of inherited server defaults. Exact inputs, readings and full-precision scores are retained in `Evidence/bench.json` [2, subsystem manifests].

B.6 Post-design seam-coefficient sensitivity

A new one-at-a-time check examines the existing seam weights. Owen01 and Yeukita01 are reduced to a maximum dimension of 1600 pixels, then passed through orientation, conditioning and line-centre estimation. The first, middle and last gaps on each page are selected by index before inspecting outputs. Eight settings share these six gaps, step limit seven and ink weight four: midpoint weight varies at step penalty 0.02, then step penalty varies at midpoint weight 0.012. This is post-design sensitivity, not a joint search or recovered historical tuning [2, `Evidence/coefficient_sensitivity.json`].

The parameterised reference matches all six unmodified default paths. Table 11 reports raw-ink intersections, total row movement and mean midpoint distance in working pixels; the midpoint is

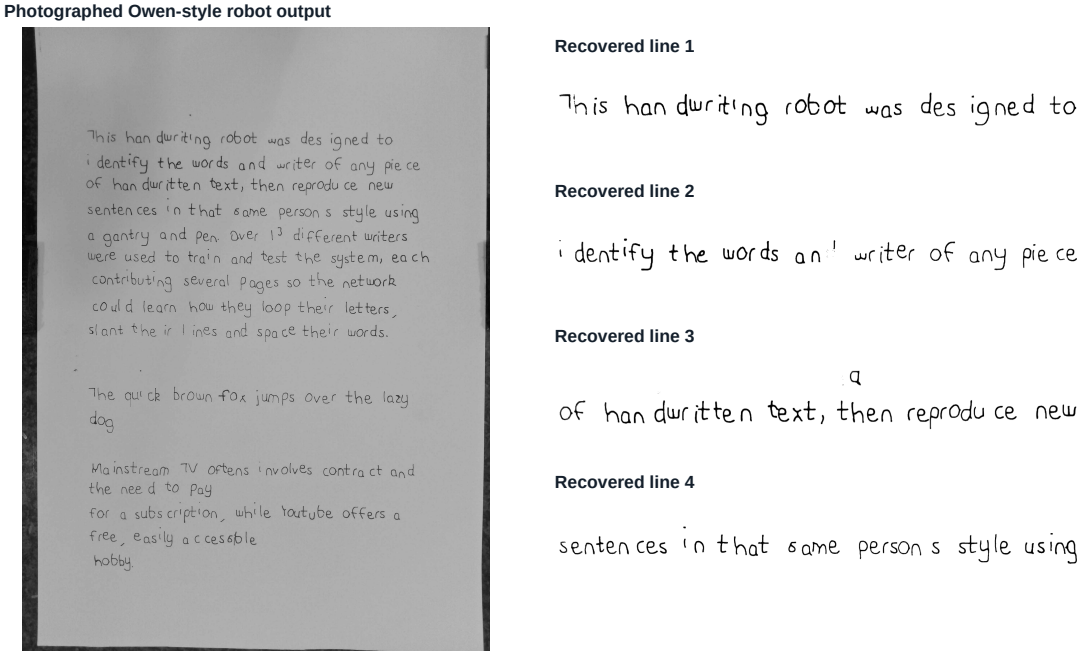


Figure 22. Existing photographed robot output and newly recovered line crops. The photograph is distinct from the software renders above. [2, 1]

a preference, not a labelled correct boundary [2, `run_coefficient_sensitivity.py`][1, segment.py, LeastCostSeam and SeamCuts].

Midpoint weight	Step penalty	Ink intersections	Row movement	Mean offset / px
0.000	0.020	2	35	4.0553
0.006	0.020	2	38	0.5517
0.012	0.020	2	42	0.3625
0.024	0.020	2	44	0.3261
0.048	0.020	2	47	0.2622
0.012	0.000	2	48	0.2622
0.012	0.010	2	44	0.3261
0.012	0.040	2	38	0.5517

Table 11. New coefficient-sensitivity check across six fixed real-page gaps; the implemented setting is 0.012/0.020 [2, `Evidence/coefficient_sensitivity.json`].

Increasing midpoint weight from zero to 0.012 reduces mean offset from 4.055 to 0.363 pixels, while row movement rises from 35 to 42 pixels. Doubling the weight gives 0.326 pixels offset. Increasing step penalty has the opposite trade-off. Every setting retains two raw-mask and three closed-mask intersections. These cases therefore show competing effects without establishing a unique optimum, eliminating intersections or measuring recognition accuracy. Input/code hashes and all 48 runs are supplied; no pipeline coefficient was changed [2, `Evidence/coefficient_sensitivity.json`].